Pérez-Jove *et al. Cybersecurity (2026) 9:84*
https://doi.org/10.1186/s42400-025-00494-y

Cybersecurity

RESEARCH

Open Access

Application of tabular transformer architectures for operating system fingerprinting

Rubén Pérez-Jove[1,2*], Cristian R. Munteanu[1,2,3], Alejandro Pazos[1,2,3] and Jose Vázquez-Naya[1,2]

Abstract

Operating System (OS) fingerprinting is essential for network management and cybersecurity, enabling accurate device identification based on network traffic analysis. Traditional rule-based tools such as Nmap and p0f face challenges in dynamic environments due to frequent OS updates and obfuscation techniques. While Machine Learning (ML) approaches have been explored, Deep Learning (DL) models, particularly Transformer architectures, owing to self-attention's ability to model complex feature dependencies, remain unexploited in this domain. This study investigates the application of Tabular Transformer architectures—specifically TabTransformer and FT-Transformer—for OS fingerprinting, leveraging structured network data from three publicly available datasets. Our experiments demonstrate that FT-Transformer generally outperforms traditional ML and DL models, previous approaches and TabTransformer across multiple classification levels (OS family, major, and minor versions). The results establish a strong foundation for DL-based OS fingerprinting, improving accuracy and adaptability in complex network environments. Furthermore, we ensure the reproducibility of our research by providing an open-source implementation.

Keywords Operating system, Fingerprinting, Identification, Detection, Deep learning, Transformer, FT-Transformer, TabTransformer, Machine learning, Cybersecurity

Introduction

The ability to accurately identify the characteristics of a host through the analysis of its network traffic is crucial for a variety of tasks in network management and computer security. Accurately identifying a machine's Operating System (OS) family and version is critical for applications including vulnerability exploitation, network inventory, and the detection of unauthorized devices.

The process of OS fingerprinting entails determining information related to the operating system—such as OS family and version—of a network-connected device by analysing its traffic. These techniques leverage differences arising from the unique ways each OS implements the communication protocol stack. Depending on the approach, OS fingerprinting can be conducted through active or passive scanning. Active scanning involves sending probes to the target and analysing responses, providing speed and reliability at the cost of a higher risk of detection. A widely used tool for this method is Nmap (Aksoy and Gunes 2016). On the other hand, passive scanning examines existing network traffic without direct interaction with the target, making it a stealthier, though generally slower and less effective, method. Tools such as p0f (Al-Shehari and Shahzad 2014) are commonly employed for passive OS detection.

*Correspondence:
Rubén Pérez-Jove
ruben.perez.jove@udc.es
[1] RNASA-IMEDIR Research Group, Department of Computer Science and Information Technologies, Facultad de Informática, Universidade da Coruña (UDC), 15071 Elviña, A Coruña, Galicia, Spain
[2] CITIC Research Centre, Universidade da Coruña (UDC), 15071 Elviña, A Coruña, Galicia, Spain
[3] IKERDATA S.L., ZITEK, University of Basque Country (UPV/EHU), Rectorate Building, 48940 Leioa, Basque Country, Spain

© The Author(s) 2026, modified publication 2026. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

Traditional rule-based approaches, as used by the aforementioned tools, are highly sensitive to variations in machine characteristics. In today's environment—characterised by a proliferation of connected devices, diverse OSs, and frequent updates—these variations pose significant challenges. An optimal solution would accurately infer a machine's OS even in scenarios where it is newly released, recently updated, or reconfigured. Artificial Intelligence (AI) techniques have demonstrated significant potential in addressing these challenges. Numerous studies have explored the application of AI to OS fingerprinting in recent years (Lastovicka et al. 2023), though most employ classical methods such as typical Machine Learning (ML) algorithms. Research on more advanced techniques, specifically Deep Learning (DL) architectures, remains limited.

Research problem. Classical rule-based and ML OS fingerprinting methods struggle to capture subtle, long-range interactions among transport- and application-layer fields. Transformer models, through multi-head self-attention, are expressly designed to learn such interactions and have delivered large accuracy gains in other structured-data tasks. This study seeks to address the problem of how to improve OS fingerprinting accuracy and robustness by leveraging advanced DL architectures—specifically, Transformer-based models designed for tabular data. By doing so, we aim to overcome the limitations of existing methods and provide a solution that is more resilient to evolving network conditions and modern OS variations.

Among recent DL architectures, the Transformer, introduced by (Vaswani et al. 2017), stands out for its ability to process sequential data efficiently through parallel processing and self-attention mechanisms. This architecture has revolutionised Natural Language Processing (NLP) with the emergence of Large Language Models (LLMs) and has been successfully adapted to other domains, including computer vision with the Vision Transformer (ViT) (Dosovitskiy et al. 2021). Its scalability and capacity for generalisation suggest that applying Transformer-based models to OS fingerprinting could yield similar breakthroughs.

A key advantage of the application of Transformers to network traffic data is their ability to capture complex interdependencies. Unlike traditional ML methods that focus on isolated features or require manual importance measures, Transformers use self-attention to process the entire data structure at once. This enables them to dynamically weigh each feature's contribution, capturing nuanced interactions essential for characterising heterogeneous, dynamic traffic, and ultimately leads to improved OS fingerprinting performance.

Contribution. In this paper, we propose the application of the Transformer architecture to OS fingerprinting. Given that network traffic data is typically stored as network flows (which can be processed as tabular data), we specifically employ a variant designed for this format: the Tabular Transformer. We analyse two variations of this architecture, namely the TabTransformer (TabT) and the FT-Transformer (FT-T), both optimised for structured tabular data processing.

To rigorously evaluate our approach, we apply it to three publicly available datasets with distinct characteristics that enable classification at multiple granular levels (OS family, major, and minor versions) under different network conditions and feature distributions. We benchmark our models against representative classical ML algorithms—k-Nearest Neighbours (kNN), Random Forest (RF), and Multi-Layer Perceptron (MLP)—and modern DL approaches—Deep Feed-forward Neural Networks (DFNN), Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM)—. Finally, we also compare our results with previous AI-based studies.

This study makes three key contributions:

- **Accuracy and robustness improvements through self-attention.** We introduce attention-based DL models for OS identification, marking the first attempt to apply Transformers—via their adaptation in Tabular Transformers—to this domain.
- **Comprehensive evaluation across multiple datasets and classification granularities.** We rigorously assess our approach using three publicly available datasets with diverse characteristics, evaluating OS classification at different levels (family, major, and minor versions) and benchmarking against classical ML models and prior research.
- **Reproducibility and transparency of experiments.** We promote further research by publicly releasing our experimental code under a GNU GPL v3.0 license. This includes preprocessing steps, model implementations, and evaluation metrics, available at: https://github.com/rubenpjove/tabularT-OS-fingerprinting.

This paper is structured as follows: Sect. 'Background' provides essential context on OS fingerprinting, ML models and the Transformer architecture; Sect. 'Related work' reviews previous works in both traditional and AI-based OS fingerprinting, including the use of Transformer to other network-related tasks; Sect. 'Materials & methods' details the experimental design and datasets used; Sect. 'Results & discussion' presents the results and comparisons with existing approaches; and

Sect. 'Conclusion & future work' summarises the findings and outlines future research directions.

Background

Operating system fingerprinting

As previously introduced, Operating System (OS) fingerprinting is the process of identifying the OS running on a network-connected device by analysing its network traffic characteristics. This technique relies on the fact that different OS implementations exhibit distinct behaviours in network communication, such as variations in Transmission Control Protocol/Internet Protocol (TCP/IP) stack parameters, packet structure, or protocol handling. By examining these characteristics, OS fingerprinting enables the classification of devices, providing valuable information such as OS family, version, or even specific configurations.

As we have already seen, OS fingerprinting plays a crucial role in both network management and security (Beddoe 2000). It allows administrators to maintain an up-to-date inventory of devices, identify and patch vulnerable systems, and detect unauthorized devices such as rogue access points or insecure personal devices. In cybersecurity, it is widely used for reconnaissance, as identifying a target's OS enables security professionals to tailor exploits to specific vulnerabilities, increasing the likelihood of a successful attack. Moreover, adversaries can leverage this information for social engineering, impersonating technical support and manipulating users into installing malicious software.

A specific example of OS fingerprinting can be achieved by analysing the Time To Live (TTL) and TCP Window Size parameters in network packets. Different OSs use distinct default values for these parameters, allowing generating specific signatures based on packet observations. Table 1 provides the relation of Window Size and TTL values for various OSs, which enables the classification process. For instance, if a network packet is observed with a TTL of 128 and a TCP Window Size of 8192, it is likely originating from a Windows 10 machine. These characteristics, combined with other parameters such as TCP options, provide valuable insights for classifying OS versions more accurately.

The level of detail that can be extracted through OS fingerprinting depends on the techniques used and the amount of network data available for analysis. When only basic network parameters, such as TTL or TCP Window Size, are available, OS identification is typically limited to broad categories, and similar OSs can be mixed, as shown in Table 1. However, incorporating more detailed protocol features allows for finer-grained classification. Depending on the level of detail in the classification, OS fingerprinting can be structured into the following levels:

- **Manufacturer:** Identifies the company or organization responsible for developing the OS, such as Microsoft or Apple.
- **Family:** Classifies the OS into a broader category or series, such as Windows, macOS, or Linux, grouping similar systems under a common architecture.
- **Major version:** Specifies the primary release version within an OS family, such as Windows 10 or macOS Catalina, indicating significant changes in functionality and system architecture.
- **Minor version:** Differentiates smaller updates, builds, or patches within a major version, such as Windows 10 version 1909 or macOS Catalina 10.15.5, allowing for more granular classification.

Based on how the scanner interacts with the network, OS fingerprinting methods are classified into two main categories: active and passive. Active fingerprinting involves sending crafted network packets to the target machine and analysing its responses. While it provides fast and accurate results, it is easily detectable and can be blocked by security mechanisms. A widely used tool for active fingerprinting is Nmap (Aksoy and Gunes 2016), but there are other examples such as Xprobe2 and SinFP (Arkin 2002; Auffret 2010; Beddoe 2000). In contrast, passive fingerprinting analyses existing network traffic without directly interacting with the target, making it stealthier but generally less accurate. This method relies on observing characteristics such as TTL, TCP Window Size, and TCP Options. Examples of passive fingerprinting tools include p0f, PRADS, and Ettercap (Zalewski 2016; Fjellskål and Wysocki 2009; Alberto et al. 2001). A

Table 1 Examples of window size and TTL values for different OSs. Source: (Lastovicka et al. 2018)

Window size	TTL	OS
8192	128	Windows 10
65,535	64	Android 6
29,200	64	Ubuntu
65,535	64	Mac OS X 10.12
65,535	64	iOS 10.3

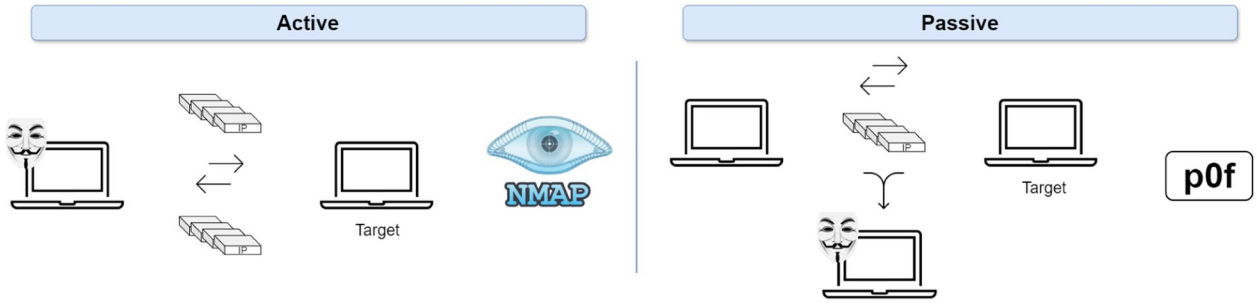


Fig. 1 Diagram of active and passive OS fingerprinting

diagram of both types of OS fingerprinting is shown in Fig. 1. Furthermore, a detailed analysis of traditional OS fingerprinting methods and tools is presented in Subsect. 'Traditional OS fingerprinting'.

While effective in many scenarios, traditional rule-based OS fingerprinting approaches face significant limitations. Frequent OS updates and patches can alter network signatures, reducing the accuracy of traditional fingerprinting techniques. Additionally, many modern systems implement security mechanisms, such as firewall rules, packet obfuscation, and OS hardening tools, which modify network responses to evade detection. These factors make it increasingly difficult to maintain up-to-date and comprehensive fingerprinting databases, limiting their ability to correctly identify newer OS versions or systems with non-standard configurations.

To overcome these challenges, AI-driven techniques, including ML and DL, have been introduced to enhance OS fingerprinting. By integrating AI techniques, modern OS fingerprinting systems improve the accuracy, handle encrypted or obfuscated traffic, and generalize better across different network conditions. The application of ML and DL to OS fingerprinting is explored in detail in Subsects. 'Machine learning-based OS fingerprinting' and 'Deep learning-based OS fingerprinting'.

Artificial intelligence

As background for this work, we provide an overview of the AI methods that form the foundation of our study. Initially, we introduce several conventional ML algorithms that serve as baselines for performance comparison. We then detail the Transformer architecture along with its specialized adaptation for structured tabular data—Tabular Transformers—which constitutes the core of our novel contribution.

Machine learning baselines

For baseline comparison, we used three established algorithms—k-Nearest Neighbors (kNN), Random Forest (RF), and Multi-Layer Perceptron (MLP)—which are representative of classical ML paradigms. From the DL field, we took as baselines the following: Deep Feed-forward Neural Networks (DFNN), Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM). These models provide a robust benchmark for evaluating performance on this task.

- **Classical ML**
  - **k-Nearest Neighbors (kNN)** (Cover and Hart 1967): A non-parametric method that assigns class labels based on the majority vote among the k closest training examples using distance metrics (e.g., Euclidean, Manhattan). Its simplicity is counterbalanced by increased computational cost on large datasets.
  - **Random Forest (RF)** (Breiman 2001): An ensemble technique that builds multiple decision trees on bootstrapped subsets with random feature selection. The final prediction is obtained by majority voting (classification) or averaging (regression), enhancing generalization and mitigating overfitting.
  - **Multi-Layer Perceptron (MLP)** (Rumelhart et al. 1986a): A feed-forward neural network comprising an input layer, one or more hidden layers, and an output layer. Trained via backpropagation, the MLP learns complex nonlinear relationships but requires careful hyperparameter tuning and entails higher computational cost.
- **DL**
  - **Deep Feed-forward Neural Networks (DFNN)** (Rumelhart et al. 1986b): A (typically) many-layer feed-forward architecture that learns hierarchical nonlinear representations via back-

propagation and (stochastic) gradient-based optimizers (e.g., SGD, Adam). Practical training relies on choices such as ReLU-like activations, careful initialization, batch normalization, and dropout to stabilize gradients and control overfitting. DFNNs excel at approximating complex functions but lack inductive biases for spatial or temporal structure and can be data-hungry.

- **Convolutional Neural Networks (CNNs)** (LeCun et al. 1989): Architectures that exploit local connectivity, weight sharing, and (often) pooling to efficiently extract translation-invariant hierarchical features from grid-structured data (e.g., images, spectrograms). Convolutions drastically reduce the number of parameters compared to fully connected layers, improving sample efficiency and scalability. CNNs dominate many computer vision tasks but are less suited to non-Euclidean or long-range sequence modeling without architectural adaptations.
- **Long Short-Term Memory (LSTM)** (Hochreiter and Schmidhuber 1997): A recurrent neural network variant designed to mitigate vanishing/exploding gradients through gated memory cells (input, forget, and output gates). Trained with backpropagation through time, LSTMs capture long-range temporal dependencies and have been widely used in language modeling, speech, and time-series forecasting. Their sequential nature limits parallelization and can incur higher computational cost compared to more recent fully parallel architectures.

The transformer architecture

Transformers (Vaswani et al. 2017) are a DL architecture specifically designed to process sequential data efficiently. Unlike traditional architectures such as Recurrent Neural Networks (RNNs) and Convolutional Neural Networks (CNNs), Transformers eliminate the need for recurrence and convolutional layers, instead leveraging an encoder-decoder structure, as depicted in Fig. 2. At the heart of this architecture lies the multi-head self-attention mechanism, which allows the model to capture global dependencies across input sequences by dynamically assigning different importance weights to each token. Self-attention naturally fuses heterogeneous categorical and numeric flow attributes, a property that classical models lack. Unlike traditional ML algorithms that rely on extensive feature engineering and domain-specific tuning, Transformers automatically learn complex relationships and interactions within the data. This capability to model long-range dependencies makes them particularly effective for sequence-to-sequence tasks.

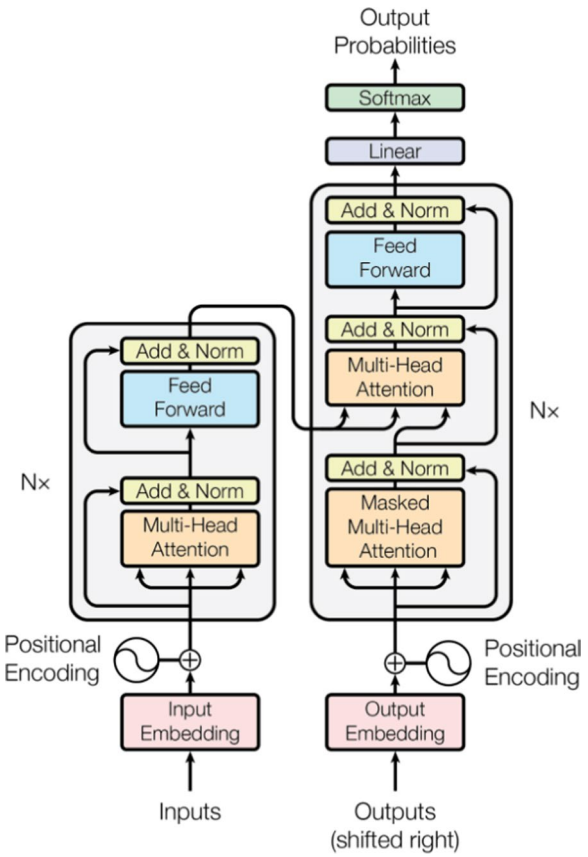


Fig. 2 Transformer architecture. Source: (Vaswani et al. 2017)

Since their introduction in 2017 by Google researchers (Vaswani et al. 2017), Transformers have revolutionized NLP, excelling in tasks such as machine translation, text summarization, and sentiment analysis. A key advantage of this architecture over RNNs is its parallel processing capability, allowing entire input sequences to be processed simultaneously rather than sequentially. This significantly reduces training time and enhances performance, particularly in capturing long-range dependencies within textual data. Consequently, Transformers have become the foundation of modern LLMs, such as OpenAI's GPT and Meta's LLaMA series, which leverage this architecture to achieve state-of-the-art performance in NLP tasks.

Beyond NLP, the adaptability of Transformers has led to their application in diverse domains, including Computer Vision (CV), Reinforcement Learning (RL), or network traffic analysis. Their ability to model complex relationships in structured data makes them suitable for various tasks, from protein structure prediction to anomaly detection in cybersecurity. The scalability and efficiency of Transformers, particularly when trained on large datasets, have contributed to their widespread

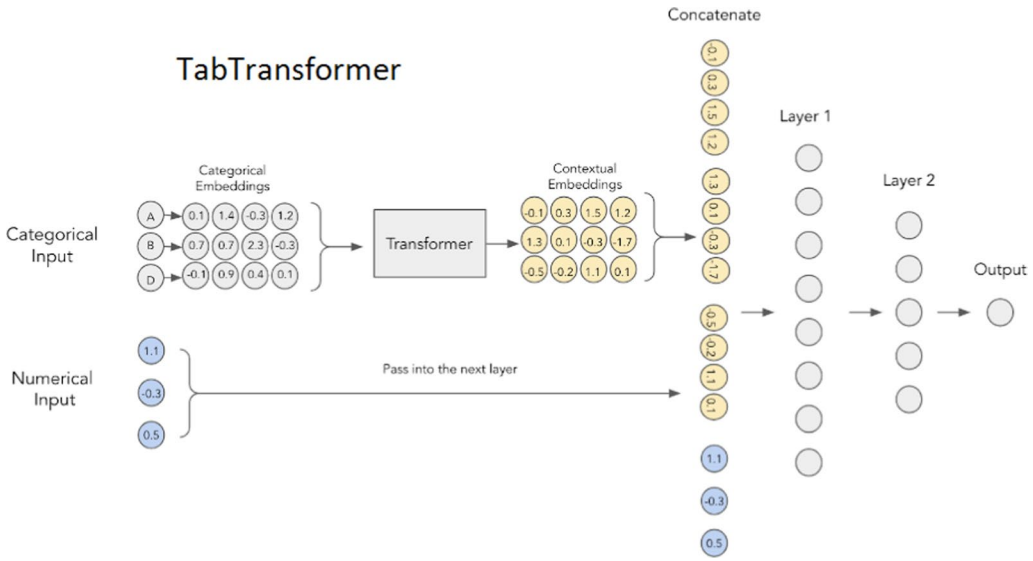


Fig. 3 Diagram of TabTransformer (TabT) architecture. Source: (Wang 2024)

adoption, setting new benchmarks across multiple fields and paving the way for their use in OS fingerprinting and network security applications.

Tabular Transformers

Tabular Transformers are an adaptation of Transformers designed to process structured tabular data commonly found in CSV files, spreadsheets, and relational databases. By leveraging self-attention mechanisms, they effectively capture complex feature interactions, eliminating the need for extensive feature engineering. These models have demonstrated superior performance in classification, regression, and other predictive tasks across various domains (Gorishniy et al. 2021).

We selected this architecture because network traffic datasets in this field are typically structured as collections of network flows, inherently formatted as tabular data. Unlike traditional DL models such as CNNs and RNNs, which rely on spatial or sequential patterns, Tabular Transformers excel at modelling structured data with heterogeneous features. Their ability to learn intricate dependencies across multiple attributes makes them particularly well-suited for analysing network traffic.

Furthermore, as discussed in Sect. 'Related work', this approach has not yet been explored for OS fingerprinting, despite its potential to enhance classification accuracy by capturing nuanced relationships in network traffic data. This gap in prior research motivated our investigation into the applicability of Tabular Transformers to this task.

Several Transformer-based architectures have been proposed for tabular data modelling. For this study, we selected two representative models—TabTransformer (TabT) and FT-Transformer (FT-T)—due to their demonstrated effectiveness in handling categorical and numerical features, allowing us to assess their suitability for OS fingerprinting. In the context of network traffic analysis, categorical features represent discrete variables with distinct groups, such as protocol type or OS family, while numerical features are continuous values that quantify measurements, like packet size or TTL.

- **TabTransformer (TabT)** (Huang et al. 2020) replaces traditional categorical embeddings with context-aware representations using Transformer layers. By applying multi-head self-attention to categorical features, it captures dependencies and interactions more effectively than standard embedding techniques, improving classification tasks and enhancing robustness to missing and noisy data.
- **FT-Transformer (FT-T)** (Gorishniy et al. 2021) generalizes the self-attention mechanism to both categorical and numerical features, treating them uniformly to better capture interactions across heterogeneous data. Unlike architectures that require extensive preprocessing or feature engineering, FT-Transformer learns feature dependencies directly from raw tabular inputs, making it well-suited for complex network traffic datasets with mixed feature types.

These models were selected to compare different feature integration strategies in OS fingerprinting. While TabTransformer processes only categorical variables through the Transformer, FT-Transformer applies self-attention to both categorical and numerical features. Figures 3, 4

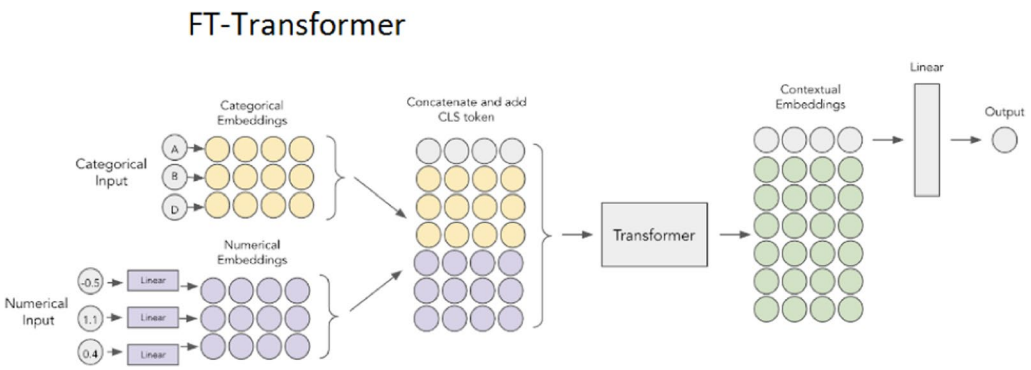


Fig. 4 Diagram of FT-Transformer (FT-T) architecture. Source: (Wang 2024)

illustrate these differences, highlighting how each architecture structures and transforms input data for prediction tasks.

Related work

OS fingerprinting

Traditional OS fingerprinting

OS fingerprinting initially emerged as an approach based solely on the analysis of TCP/IP header fields, such as Time To Live (TTL), the Don't Fragment (DF) flag, Type of Service (ToS) and TCP Window Size (Aksoy and Gunes 2016) (Lastovicka et al. 2023). In this context, Nmap was one of the first tools to be developed and remains one of the most widely used, as it employs an active scanning method that compels the target machine to respond, thereby facilitating the identification of its operating system.

Alternatively, passive fingerprinting techniques were subsequently proposed, relying on similar network traffic characteristics but without provoking a response from the target system. Early implementations of this approach, such as p0f and Siphon, emerged in 2000 (Hagos et al. 2021; Zalewski 2016). As previously discussed (Sect. 'Operating system fingerprinting'), the fundamental difference between the two methods lies in the manner in which network information is collected, rather than in the analytical techniques applied to the traffic for inference.

Traditional OS fingerprinting methods based on TCP/IP headers continued to evolve. Tools like Ettercap and Satori extended earlier approaches (Alberto et al. 2001; Kollmann 2018), while others like NetSleuth and PRADS saw limited longevity (NetGrab 2012; Fjellskål and Wysocki 2009). Research expanded into passive OS fingerprinting in large networks, examining features from network flows, as demonstrated by Vymlátil (2014); Matousek and Wagner (2014). These methods proved effective in dynamic environments such as wireless networks, and newer approaches—such as those by Al-Shehari and Shahzad (2014); Osanaiye and Dlodlo (2015)—combined traditional methods with ML to achieve better accuracy, particularly in unauthorized OS detection and cloud environments.

Modern approaches have shifted towards analysing application layer protocols, encrypted traffic, and specialised traffic types. Hypertext Transfer Protocol (HTTP) banners and User-Agent strings provide more precise OS identification (Shah 2003), while encryption complicates traditional methods. Researchers like Matousek and Wagner (2014); Fan et al. (2019) improved accuracy by incorporating Transport Layer Security (TLS) handshake features, and others like Aksoy and Gunes (2016) explored various protocols using ML to optimise fingerprinting. In specific cases, such as smartphone OS identification and Industrial Control System (ICS) devices, timing analysis and advanced algorithms have been used Gurary et al. (2016); Shen et al. (2018). ML has become essential in overcoming traditional limitations, focusing on processing large datasets and achieving higher accuracy in identifying OS in encrypted traffic (Beverly 2004; Shamsi et al. 2021; Lastovicka et al. 2018). A detailed explanation of the works where ML is applied to the OS fingerprinting field is exposed in Sect. 'Related work'.

Machine learning-based OS fingerprinting

The field of OS fingerprinting has evolved significantly with the advent of ML techniques. In this context, several approaches have been proposed to enhance the accuracy and robustness of OS identification. For instance, Lastovicka's research (Lastovicka et al. 2018) explored various classical ML algorithms like Naive Bayes (NB), Decision Trees (DT), k-Nearest Neighbours (kNN), and Support Vector Machines (SVM) for passive OS fingerprinting. In a later study (Lastovicka et al. 2020), Lastovicka et al. expanded this work by employing TLS

handshake features, which improved device identification even in encrypted network environments. Similarly, Fan et al. (2019) employed Gradient Boosting Decision Trees (GBDT) on features extracted from both TLS and TCP/IP headers, achieving high accuracy on a large dataset. This aligns with previous contributions made by our research team (Pérez-Jove et al. 2021, 2023), where we applied a range of classical ML algorithms like NB, Multilayer Perceptron (MLP), DT, and Random Forest (RF) to the Nmap and p0f databases.

Other studies focused on minimalist data requirements and novel feature extraction. Millar et al. (2020) used RF to classify OS types based on IP affiliation graphs, demonstrating resilience to encrypted traffic. Similarly, Barath and Liska (2021) employed DT, Expectation-Maximization (EM), NB, and Artificial Neural Networks (ANNs) for passive monitoring, further showcasing the potential of various ML techniques for network data analysis. Shamsi et al. (2021) also used a non-parametric EM estimator to improve OS fingerprinting accuracy in noisy data for large-scale network environments and dealing with distortions.

Several works focused on specialised environments or different network protocols. Salah et al. (2022) focused on IPv6-based fingerprinting using kNN, DT, SVM, and Gaussian Naive Bayes (GNB), whereas Bub et al. (2022) applied DT to identify aged Android devices in home networks. Hulák et al. (2023) compared the performance of DT, RF, and AdaBoost (AB) in passive OS fingerprinting, emphasising the importance of careful data preparation. In a related domain, Zhang et al. (2022) integrated Active Learning (AL) with SVM, RF, and NB to optimise OS identification, particularly in environments with dynamic network conditions.

The literature review efforts in this field are limited, with Lastovicka et al. (2023) providing the only comprehensive survey of passive OS fingerprinting methods, detailing the transition from traditional techniques to ML-based approaches.

Deep learning-based OS fingerprinting

Even though ML techniques have been explored in OS fingerprinting, as previously outlined, there is little research on the application of DL models to this field. For the best of author's knowledge, only two works employed some DL algorithm to solve this network task. Li et al. (2023) proposed a combined sampling method paired with a Convolutional Neural Network (CNN) to improve identification accuracy for underrepresented OS types in imbalanced datasets. Hagos et al. (2020, 2021) introduced the TCP variant as a feature in ML models, exploring a mix of traditional ML algorithms, like SVM, RF, kNN, NB, with DL models like Long Short-Term Memory (LSTM). Finally, a preliminary version of this work was presented in Pérez Jove et al. (2024), where the Transformer architecture was applied to the Nmap database.

Transformers in network traffic modelling

Beyond its success in NLP, the Transformer architecture has proved versatile across domains such as computer vision (e.g., the Vision Transformer (ViT) (Han et al. 2023)) and network analysis. Although Transformers have not yet been applied directly to OS fingerprinting, they have been adapted for various networking tasks. For example, NetBERT outperforms BERT on network-specific tasks (Louis and Louppe 2020), while adaptations of BERT for DNS analysis and the use of Graph Neural Networks for packet sequences have also been explored (Le et al. 2022). Direct training on network traffic has yielded promising results too, with the Residual 1-D Image Transformer excelling in malware and DDoS detection (Barut et al. 2023) and the Flow Transformer enhancing anonymity network classification by capturing temporal-spatial dependencies (Zhao et al. 2021). Furthermore, Torre Vico et al. (2024) have demonstrated the potential of LLMs in analysing network traces for cybersecurity.

Concurrently, interest in Network Traffic Foundational Models (NT-FMs) inspired by large language models is growing. Early work includes ET-BERT, which leverages contextualised datagram representations for encrypted traffic classification (Lin et al. 2022), and Ray's packet-level traffic prediction model (Ray 2022). Subsequent studies have investigated model generalisation (Dietmüller et al. 2022) and foundational applications in networking (Le et al. 2022). More recent advances include Zhao et al.'s Yet Another Traffic Classifier (YATC) using a masked autoencoder with multi-level flow representations (Zhao et al. 2023), Guthula et al. netFound utilising unlabelled traffic for pre-training (Guthula et al. 2023), and Wang et al. Lens capturing temporal-spatial correlations for anomaly detection (Wang et al. 2024). Additional contributions include TrafficGPT for long traffic sequence modelling (Qu et al. 2024), a graph-based NT-FM by Langendonck et al. for improved scalability (Van Langendonck et al. 2024), the generative pretrained model NetGPT for traffic understanding and generation (Meng et al. 2023), and the comprehensive NetBench dataset for evaluating foundational models on traffic tasks (Qian et al. 2024).

Materials & methods

Datasets

For evaluating OS fingerprinting methods, we selected publicly available benchmark datasets that meet essential

criteria for evaluating OS fingerprinting methods. Specifically, datasets must have sufficient size, diversity, and accurate OS labelling, and given the rapid evolution of OSs, they need to be up-to-date. Our selection process was informed by their adoption in recent studies and an analysis of prior works (Sect. 'Related work'), ensuring both relevance and comparability. Moreover, these datasets capture network traffic at various abstraction levels—from high-level flow summaries to detailed packet-level data—and provide OS labels at multiple granular levels (family, major, and minor versions).

We selected three representative datasets—`DAT1`, `DAT2`, and `DAT3`—that capture diverse network features essential for robust OS fingerprinting. In our review of prior works, we analysed available features like packets, telemetry, and logs to identify datasets that are both recent and reflective of realistic network environments with current OS versions and modern traffic dynamics. These datasets encompass diverse data types (flow-level records, packet captures, and active OS fingerprinting signatures), offer multiple levels of OS granularity (family, major, and minor versions), vary in size, and originate from different network settings. This diversity underpins a robust evaluation of AI-based OS fingerprinting models by mitigating dataset bias and enhancing generalisability.

Three complementary data types—IPFIX, PCAP, and OS signature databases— were employed to comprehensively analyse network traffic for OS fingerprinting. Specifically, IPFIX (Internet Protocol Flow Information Export) provides flow-level metadata summarizing network activity; PCAP (Packet CAPture) retains raw network packets to capture detailed protocol behaviour; and OS signature databases, such as those used by Nmap, offer predefined OS fingerprints as classification references. Together, these sources enable a multifaceted analysis that integrates both high-level traffic patterns and in-depth protocol details.

We defined OS classification tasks at three granularity levels—`family`, `major`, and `minor`—to capture varying levels of detail. Specifically, the `family` level includes broad categories (e.g., Windows, Linux, Android), the `major` level distinguishes versions (e.g., Windows 10, Android 9, iOS 13), and the `minor` level provides finer distinctions (e.g., Windows 8.1, iOS 13.5, macOS 10.15). For instance, a detailed classification might separate `Ubuntu` into `22.04 (Jammy Jellyfish)` and further into `22.04.3 LTS`, whereas a less detailed approach would label it simply as `Ubuntu`. Higher granularity, particularly at the `minor` level, increases classification complexity due to a larger number of classes and fewer training examples per class. The characteristics of the selected datasets are further detailed in the following points and summarized in Table 2.

- **DAT1**: **lastovicka_2019_UsingTLS** (Martin et al. 2019)

  This dataset comprises flow records from the Czech Republic Masaryk university's backbone network, enriched with log entries from Dynamic Host Configuration Protocol (DHCP) servers and a Remote Authentication Dial-In User Service (RADIUS) accounting server. Collected between July 12 and 16, 2019, it focuses on flows originating from the university's Eduroam wireless networks. OS labels are derived from DHCP logs and RADIUS session IDs.

  Useful features for OS fingerprinting include basic flow attributes, extended TCP/IP parameters, HTTP user-agent strings, and TLS client details. The dataset, anonymized using the Crypto-PAn algorithm, spans 18.7 million rows with 29 features, primarily supporting OS family-level classification with five classes.
- **DAT2**: **lastovicka_2023_PassiveOSRevisited** (Lastovicka et al. 2023)
  This dataset captures web traffic from five Masaryk university servers hosting 475 domains over eight hours. OS labels are derived from HTTP User-Agent strings in web server logs, cross-referenced with network flow data. Collected connections include devices such as user computers, mobile phones, and web crawlers.
  OS fingerprinting-related features include IP and TCP parameters, HTTP and TLS details, among others, amounting to 112 features in total. The dataset includes 109,663 rows, enabling OS classification at family (12 classes), major (50 classes), and minor (88 classes) levels.
- **DAT3**: **nmap–7.94_2023_OSdb** (Zhang et al. 2022)
  This dataset consists of OS signatures actively collected using the Nmap tool (version 7.94). Nmap identifies OSs by sending 16 TCP, User Datagram Protocol (UDP), and Internet Control Message Protocol (ICMP) probes and analysing responses. Features include TCP window sizes, sequence generation, and TCP options, providing detailed fingerprinting data.
  The dataset contains 38,817 rows with 263 features and supports fine-grained OS classification at family (7 classes) and minor (91 classes) levels. Its active collection process complements the passive data in `DAT1` and `DAT2` by capturing precise protocol behaviours.

Class distribution analysis revealed significant imbalances across all datasets, meaning that some classes contain far more examples than others. This imbalance,

Table 2 Overview of the employed datasets

Code	Year	Works	Format	Feature count						Examples	Class	Count
				Total	TCP/IP	DNS	HTTP	TLS	Other			
`DAT1` (Martin et al. 2019)	2019	(Lastovicka et al. 2020)	IPFIX	29	7	–	5	8	9	18,708,983	`family`	5
`DAT2` (Lastovicka et al. 2023)	2023	(Lastovicka et al. 2023; Hulák et al. 2023)	IPFIX	112	35	–	7	28	46	109,663	`family`	12
											`major`	50
											`minor`	88
`DAT3` Zhang et al. (2022)	2023	(Pérez-Jove et al. 2021; Pérez Jove et al. 2024)	DB	263	263	–	–	–	–	38,817	`family`	7
											`minor`	91

illustrated in Fig. 5, introduces challenges for training, as models may become biased toward majority classes. Addressing this imbalance is critical to achieving robust and fair performance across all tasks, and will be discussed in Sect. 'Data preparation'.

It's worth mentioning that the OS annotations in the three benchmarks stem from prior rule-based or classical ML procedures: `DAT1` combines DHCP and RADIUS logs, `DAT2` derives labels from HTTP *User-Agent* strings, and `DAT3` relies on Nmap's probe-signature matching. While these mechanisms are well established, they inevitably inherit the limitations of their underlying heuristics and may therefore contain residual noise. Nonetheless, the datasets remain de-facto public benchmarks for OS fingerprinting and are widely adopted in recent studies; we employ them to ensure comparability with the State of the Art (SoA).

Data preparation

Data preparation is the process of cleaning, transforming, and balancing datasets to ensure robust model performance and reproducibility. In our study, this involves handling missing or invalid data, removing irrelevant features, and addressing class imbalances—each with fixed parameters and random seeds to guarantee full reproducibility.

Handling missing data and redundancies. We improve data quality by systematically addressing missing values and redundant entries. In the datasets, Not a Number (NaN) or Null values—indicating undefined or unrepresentable quantities (e.g., missing TLS features in unencrypted flows)—are encoded categorically when applicable; rows with missing values in critical columns are removed, and numerical features with zero variance (i.e., variance ≤ 0) are dropped. Duplicate entries are also

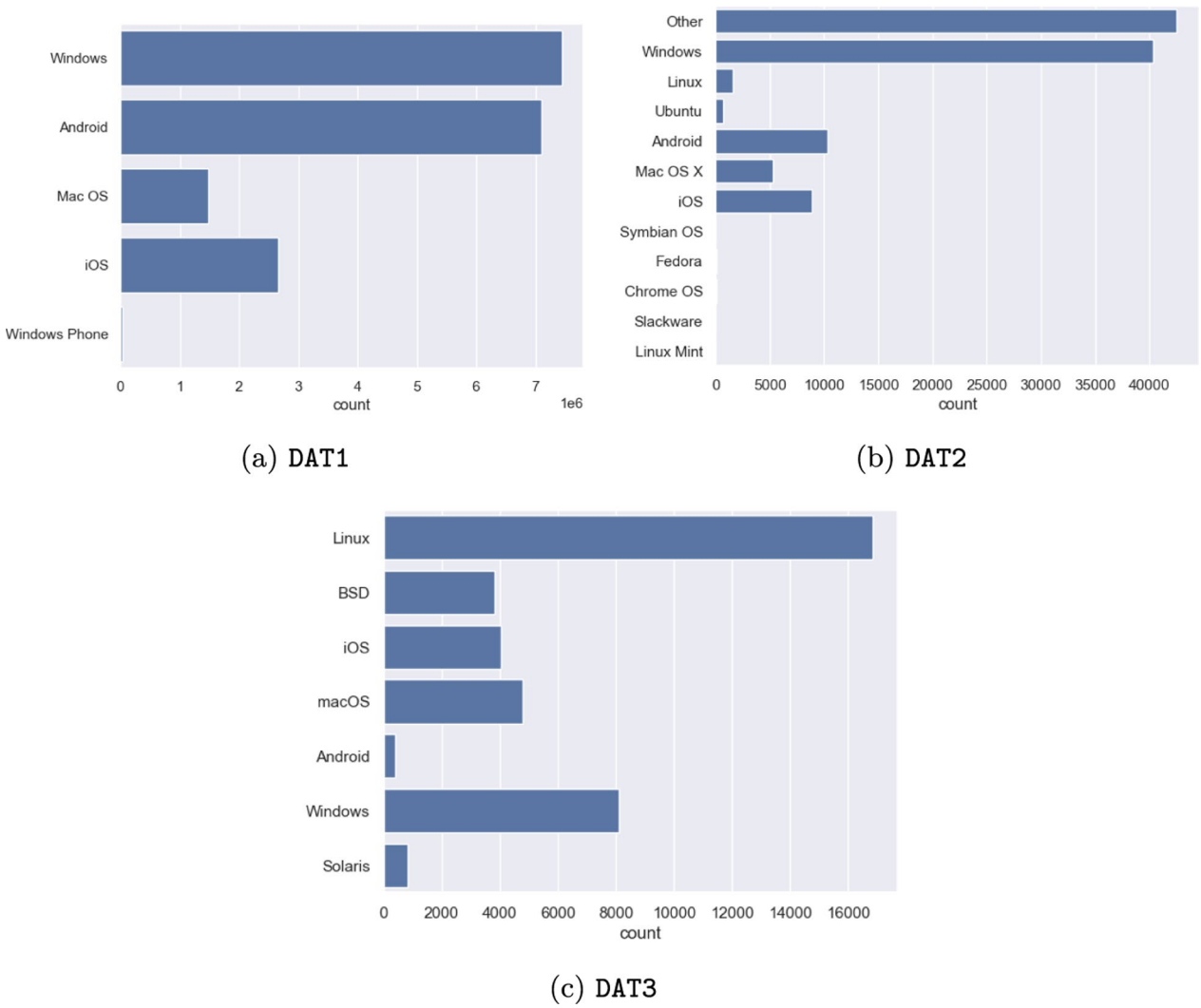


Fig. 5 Class distribution by OS `family` version in `DAT1` (*top-left*), `DAT2` (*top-right*), and `DAT3` (*bottom-centre*)

removed to avoid redundancy, and no other errors or infinite values were detected.

Removing irrelevant features. Irrelevant columns are removed to focus on OS fingerprinting. For example, columns related to timestamps or non-OS network information—such as "`Date flow start`" and "`Session ID`" in `DAT1`—are excluded, with similar removals performed in `DAT2` and `DAT3`. The precise selection of retained features is provided in Table 3, along with explicit lists of both categorical and numerical features to standardize the preprocessing pipeline.

Addressing class imbalances. We mitigate class imbalances by standardizing target classes and applying resampling techniques. Target classes are first standardized via regular expression matching (e.g., mapping entries containing "iOS", "Android", "Mac OS X", and "Windows" to their respective labels). Then, random undersampling is applied to majority classes using predetermined removal percentages (with fixed random seeds), while the Synthetic Minority Over-sampling Technique (SMOTE) is employed with a sampling strategy of "`auto`" to generate synthetic samples for minority classes:

1. **Random undersampling:** Applied to majority classes when sufficient data is available.
2. **SMOTE:** Employed to generate synthetic samples for minority classes.

Feature processing and encoding. Feature processing further refines both categorical and numerical data for improved interpretability and balanced training. Categorical columns containing hexadecimal strings are split into individual bytes, and One-Hot Encoding is applied to categorical target variables—converting them into a numerical format by creating binary columns for each class, after which the original target column is removed. Additionally, class weights are computed as the inverse of normalized class frequencies, and data is split into training and test sets using stratified sampling (with 20% reserved for testing).

Reproducibility and software environment. Reproducibility is ensured by using fixed versions of key libraries and by sharing the complete code. Our experiments rely on `numpy==1.23.0`, `pandas==2.2.2`, `scikit-learn==1.5.0`, `torch==2.3.1`, and `optuna==3.6.1` for data manipulation, model training, and hyperparameter optimization. The full code implementing these steps is available at https://github.com/rubenpjove/tabularT-OS-fingerprinting.

Modelling

We present a comprehensive, reproducible, and efficient approach to model selection, training, and validation. After preparing each dataset, we first split the data into training and testing sets using stratified sampling to maintain class distribution—this same strategy is applied within cross-validation splits. The training set is used exclusively for model development, while the test set is reserved for final performance evaluation. To obtain optimal hyperparameters for the TabTransformer (TabT) (Huang et al. 2020) and FT-Transformer

Table 3 Selected features for each dataset

Dataset	Numerical features	Categorical features
DAT1	TLS Client Version, Client Cipher Suites, TLS Extension Types, TLS Extension Lengths, TLS Elliptic Curves, TLS EC Point Formats	SYN size, TCP win, TCP SYN TTL
DAT2	TCP flags A, TLS_CONTENT_TYPE, TLS_HANDSHAKE_TYPE, TLS_CIPHER_SUITE, TLS_CLIENT_VERSION, TLS_CIPHER_SUITES, TLS_CLIENT_SESSION_ID, TLS_EXTENSION_TYPES, TLS_CLIENT_KEY_LENGTH, TLS_EXTENSION_LENGTHS, TLS_ELLIPTIC_CURVES, TLS_EC_POINT_FORMATS, IPv4DontFragmentforward, tcpOptionWindowScaleforward, tcpOptionSelectiveAckPermittedforward, tcpOptionNoOperationforward, flowEndReason, TLS_JA3_FINGERPRINT, IP ToS	SRC port, TCP SYN Size, TCP Win Size, TCP SYN TTL, NPM_CLIENT_NETWORK_TIME, NPM_ROUND_TRIP_TIME, NPM_RESPONSE_TIMEOUTS_A, NPM_TCP_RETRANSMISSION_A, NPM_TCP_OUT_OF_ORDER_A, NPM_JITTER_DEV_A, NPM_JITTER_AVG_A, NPM_JITTER_MIN_A, NPM_JITTER_MAX_A, NPM_DELAY_DEV_A, NPM_DELAY_AVG_A, NPM_DELAY_MIN_A, NPM_DELAY_MAX_A, NPM_DELAY_HISTOGRAM_1_A, TLS_SETUP_TIME, tcpOptionMaximumSegmentSizeforward
DAT3	SEQ.SP, SEQ.GCD, SEQ.ISR, SEQ.TI, SEQ.CI, SEQ.II, SEQ.TS, WIN.W*, ECN.T, ECN.TG, ECN.W, T*.T, T*.TG, T*.RD, T*.W, U1.T, U1.TG, U1.IPL, U1.UN, U1.RIPL, U1.RID, U1.RUCK, IE.T, IE.TG, IE.CD	SEQ.TI, SEQ.CI, SEQ.II, SEQ.SS, SEQ.TS, OPS.O1, OPS.O2, OPS.O3, OPS.O4, OPS.O5, OPS.O6, ECN.R, ECN.DF, ECN.O, ECN.CC, ECN.Q, T*.R, T*.DF, T*.S, T*.A, T*.F, U1.R, U1.DF, U1.RIPL, U1.RID, U1.RIPCK, U1.RUCK, U1.RUD, IE.R, IE.DFI, IE.CD

(FT-T) (Gorishniy et al. 2021), we conducted a hyperparameter search using Optuna's NSGA-II sampler (Table 4). In each trial, key metrics such as training time, inference time, model parameter count, and memory usage were recorded as user attributes to enhance reproducibility and analysis. Notably, all random seeds are fixed to guarantee reproducibility, and our approach is computationally more efficient than traditional methods like Grid or Random Search.

Hyperparameter search trials. In each trial, we employ Stratified 10-Fold Cross-Validation with resampling techniques to handle class imbalance and optimize model performance. Specifically, data is split into training and validation sets with balanced class distributions across all folds. For class imbalance, random undersampling (with explicit removal percentages for majority classes) and SMOTE (with a fixed random state) are applied. Each fold undergoes 200 training epochs with batch sizes of 128, 256, or 512, and early stopping is triggered after 15 epochs without improvement. Model parameters are updated using the AdamW optimizer and training is guided by Cross-Entropy Loss. This entire process is executed on a Compute Unified Device Architecture (CUDA)-enabled device, utilizing data parallelization and custom Graphics Processing Unit (GPU) memory management (in conjunction with garbage collection) to ensure efficient computations and prevent memory leaks.

Baseline models. We also trained several baseline models, representing relevant alternatives of classical ML algorithms—k-Nearest Neighbors (kNN), Random Forest (RF), and Multi-Layer Perceptron (MLP)—and DL—Deep Feed-forward Neural Networks (DFNN), Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM)—, to provide a comparative performance analysis. For these models, the same preprocessing pipeline was applied, except that categorical features were encoded using One-Hot Encoding (classical ML models) or Embeddings (DL). Each baseline model was trained with a set of default hyperparameters (see Table 5) and evaluated using Stratified 10-Fold Cross-Validation, with performance metrics averaged across folds. Detailed descriptions of these baseline models are provided in Sect. 'Machine learning baselines'.

Computational resources

This research leveraged the FinisTerrae III supercomputer at CESGA (Zhao et al. 2021) for model training. This systems is a Bull ATOS bullx configured across 13 racks, which includes 714 Intel Xeon Ice Lake 8352Y processors and 157 GPUs (141 Nvidia A100 and 16 Nvidia T4 units). It has 126 TB of memory, 359 TB of SSD NVMe storage, and Infiniband HDR 100 for networking, achieving a peak performance of 4.36 PetaFLOPS. Different hardware configurations were used for the experiments, based on the computational requirements of each task and the availability within the system's job scheduling system.

Evaluation

This section details our comprehensive evaluation framework designed to rigorously assess the performance and efficiency of the proposed Tabular Transformer models for OS fingerprinting.

After hyperparameter optimization, the best-performing Tabular Transformer model was retrained on the full training dataset and evaluated on a hold-out test set using the same preprocessing pipeline. Early stopping was employed during training to prevent overfitting, and all splits were generated via stratified sampling to preserve the class distribution inherent to the OS fingerprinting task.

The evaluation metrics included are accuracy, precision, recall, and F1-score. For the latest three, as we are evaluating a multiclass classification problem, we employed the weighted average technique. These metrics, defined in the following Subsect. 'Evaluation metrics definitions', provide a robust assessment of the methods' performance. The accuracy metric itself can be very misleading on

Table 4 Hyperparameters values

Hyperparameter	Value	Description
`learning_rate`	[0.0001–0.1]	The range of learning rates used during training.
`embedding_dim`	[16, 32]	The dimensionality of the embeddings for categorical features.
`depth`	[2–6]	The range for the number of Transformer layers.
`heads`	[2–8]	The range for the number of attention heads in each transformer layer.
`attn_dropout`	[0.05–0.5]	The range for the dropout rate for the attention mechanism.
`ff_dropout`	[0.05–0.5]	The range for the dropout rate for the feedforward network within each Transformer layer.
`use_shared_categ_embed`	[True, False]	Determines whether to use shared embeddings for categorical features. Applicable to TabTransformer only.

Table 5 Default hyperparameters for classical ML and DL baselines

Model	Hyperparameters
kNN	`leaf_size`: 30
	`metric`: minkowski
	`n_neighbors`: 5
	`weights`: uniform
RF	`n_estimators`: 100
	`criterion`: gini
	`max_depth`: None
	`min_samples_split`: 2
	`min_samples_leaf`: 1
MLP	`hidden_layer_sizes`: [100]
	`activation`: ReLU
	`solver`: adam
	`learning_rate`: 0.0001
DFNN	`learning_rate`: 0.001
	`emb_dim`: 16
	`hidden_sizes`: [128, 128, 128]
	`activation`: ReLU
CNN	`learning_rate`: 3e-4
	`weight_decay`: 1e-2
	`emb_dim`: 16
	`conv1`: out_channels=32, kernel_size=`input_length`, padding=(`input_length`-1)/2
	`pool`: MaxPool1d(kernel_size=2)
	`fc1`: 64
LSTM	`learning_rate`: 0.001
	`hidden_dim`: 32
	`num_layers`: 2
	`dense_dim`: 100
	`activation`: ReLU

imbalanced datasets where one target class dominates the dataset. Furthermore, as when it comes to evaluate OS fingerprinting, we want to have a good balance between precision (how accurate the model is in its positive predictions) and recall (how complete the model's positive predictions are). Therefore, the metric we want to focus when comparing different results is F1-score, which is a harmonic mean of both.

In addition to accuracy metrics, we recorded the total training time and measured inference time, while also computing key model characteristics such as the number of trainable parameters and the overall memory footprint.

Furthermore, confusion matrices were generated to analyse misclassification at the class level. Final predictions along with their corresponding ground truth labels were saved to a CSV file, and the list of class labels was written to a separate text file. These steps ensure that the evaluation results are fully reproducible and facilitate subsequent analyses of the model's performance on an imbalanced, multiclass OS fingerprinting problem.

Evaluation metrics definitions

Our evaluation relies on the following metrics:

Accuracy measures the overall correctness of predictions:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Balanced accuracy computes the average recall per class, ensuring equal contribution from all classes:

$$\text{Balanced Accuracy} = \frac{1}{C} \sum_{i=1}^{C} \frac{TP_i}{TP_i + FN_i}$$

where C is the number of classes.

Precision quantifies the proportion of true positives among all positive predictions:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall (Sensitivity) measures the proportion of true positives identified among all actual positives:

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1-Score is the harmonic mean of Precision and Recall:

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Weighted averages are computed to account for class imbalance by weighting each class metric by its support:

$$\text{Weighted Metric} = \frac{\sum_{i=1}^{C} (\text{Support}_i \times \text{Metric}_i)}{\sum_{i=1}^{C} \text{Support}_i}$$

Results & discussion

Experimental results

Our experiments demonstrate that Transformer-based models, particularly FT-Transformer, generally outperform classical ML methods in OS fingerprinting tasks across various datasets and classification granularities. In this work, we applied two Transformer architectures—TabTransformer (TabT) and FT-Transformer (FT-T)—to three distinct datasets featuring OS information at different levels (from broad family categories to detailed major and minor versions) and compared their performance with traditional models (kNN, RF, MLP).

Hyperparameter optimization. Optimal hyperparameters for each Tabular Transformer architecture

were identified (see Table 6). Using these parameters, we trained the models and evaluated them on the different datasets and classification tasks.

- **DAT1**: Only OS `family` classification was feasible. FT-Transformer significantly outperformed Tab-Transformer and traditional models (kNN, RF, MLP) in all computed metrics, as shown in Table 7.
- **DAT2**: This dataset supported `family`, `major`, and `minor` classifications. For `family` classification, the Random Forest (RF) model marginally outperformed the Transformer-based models; however, for both `major` and `minor` levels, FT-Transformer achieved the highest performance (see Table 8).
- **DAT3**: With `family` and `minor` classifications available, FT-Transformer led in the `family` category, while TabTransformer showed superior performance for the `minor` level (refer to Table 9).

The key distinction between TabT and FT-T is how each architecture treats heterogeneous features. FT-T feeds both categorical and numerical tokens into its self-attention layers, letting it capture cross-type dependencies (e.g. `User-Agent bytes` / `TCP timing`). TabT embeds only categoricals and concatenates raw numericals after attention, so these long-range links are lost. The gap is most visible on `DAT2-major` (F1: 79.32 vs 74.99) where subtle latency fields are decisive; conversely, on `DAT3-minor` the information is almost entirely categorical, so TabT regains a slight edge.

The confusion matrices shown in Figs. 7, 8, 9 reveal systematic, not random, errors:

- (i) On `DAT1` both Transformers confuse *Windows Phone* with *iOS* and *macOS* with *Windows*, reflecting the tiny support of those classes (1-2% of flows) and their nearly identical TLS cipher-suite sets.

Table 6 Best hyperparameter combinations for each type of Tabular Transformer, classification task and dataset

Dat.	Class.	Model	Hyperparameter						
			`l_rate`	`e_dim`	`depth`	`heads`	`attn_d`	`ff_d`	`shared_categ`
DAT1	family	TabT	0.00010	16	5	8	0.05	0.20	False
		FT-T	0.00198	32	2	4	0.15	0.25	True
DAT2	family	TabT	0.00072	32	4	2	0.15	0.50	True
		FT-T	0.00105	32	6	4	0.35	0.45	True
	major	TabT	0.00133	16	4	2	0.05	0.05	True
		FT-T	0.00105	32	6	4	0.35	0.45	True
	minor	TabT	0.00072	16	2	8	0.15	0.10	False
		FT-T	0.00208	32	3	6	0.25	0.15	True
DAT3	family	TabT	0.00039	16	2	6	0.05	0.50	True
		FT-T	0.00198	32	2	4	0.15	0.25	True
	minor	TabT	0.00028	16	3	4	0.45	0.25	False
		FT-T	0.00208	32	3	6	0.25	0.15	True

Bold means the best result for that metric for a specific experiment (class/dataset)

Table 7 Metrics results for experiments in `DAT1`

	Model	Metric				
		Accuracy (%)	Balanced Acc. (%)	Precision (%)	Recall (%)	F1-score (%)
family	kNN	59.28	56.17	59.99	59.28	58.11
	RF	88.42	78.26	87.57	88.42	87.93
	MLP	89.96	80.61	89.68	89.96	89.45
	DFNN	86.71	84.10	89.50	86.71	87.63
	CNN	87.74	**85.20**	89.73	87.74	88.39
	LSTM	76.87	76.96	81.28	76.87	76.79
	TabT	89.20	82.87	89.96	89.20	89.46
	FT-T	**90.69**	83.41	**91.01**	**90.69**	**90.80**

Bold means the best result for that metric for a specific experiment (class/dataset)

Table 8 Metrics results for experiments in DAT2

	Model	Metric				
		Accuracy (%)	Balanced Acc. (%)	Precision (%)	Recall (%)	F1-score (%)
family	kNN	91.60	90.28	92.54	91.60	91.89
	RF	**95.35**	92.45	**95.52**	**95.35**	**95.33**
	MLP	93.99	90.78	94.13	93.99	93.99
	DFNN	94.62	92.10	94.74	94.62	94.66
	CNN	94.01	91.94	94.38	94.01	94.13
	LSTM	94.18	91.79	94.42	94.18	94.26
	TabT	93.86	91.90	94.10	93.86	93.93
	FT-T	95.04	**93.47**	95.19	95.04	95.09
major	kNN	70.41	71.89	75.81	70.41	71.79
	RF	73.28	66.93	**89.86**	73.28	75.64
	MLP	73.59	67.41	78.50	73.59	75.10
	DFNN	76.37	73.74	78.12	76.37	76.82
	CNN	73.87	74.54	77.46	73.87	74.63
	LSTM	75.44	74.29	77.95	75.44	76.05
	TabT	74.46	72.07	76.25	74.46	74.99
	FT-T	**79.09**	**80.28**	80.00	**79.09**	**79.32**
minor	kNN	58.92	62.73	67.65	58.92	60.63
	RF	61.80	57.48	**86.58**	61.80	65.70
	MLP	61.52	58.27	70.50	61.52	63.92
	DFNN	65.12	65.97	70.93	65.12	66.30
	CNN	60.93	65.40	69.66	60.93	62.55
	LSTM	62.54	65.85	70.17	62.54	64.02
	TabT	67.74	63.50	69.49	67.74	68.28
	FT-T	**68.52**	**69.31**	72.66	**68.52**	**69.76**

Bold means the best result for that metric for a specific experiment (class/dataset)

Table 9 Metrics results for experiments in DAT3

	Model	Metric				
		Accuracy (%)	Balanced Acc. (%)	Precision (%)	Recall (%)	F1-score (%)
family	kNN	91.00	92.31	91.23	91.00	91.05
	RF	91.64	92.54	92.37	91.64	91.62
	MLP	91.65	92.70	93.50	91.65	91.62
	DFNN	91.72	92.49	93.31	91.72	91.58
	CNN	90.79	91.76	92.63	90.79	90.66
	LSTM	90.83	91.88	92.69	90.83	90.84
	TabT	91.45%	92.27%	91.64%	91.45%	91.43%
	FT-T	**92.35**	**93.15**	**93.74**	**92.35**	**92.23**
minor	kNN	74.42	63.49	76.68	74.42	74.92
	RF	73.92	60.19	81.76	73.92	75.66
	MLP	72.94	62.19	75.89	72.94	74.14
	DFNN	74.46	**64.72**%	**81.87**%	74.46	74.89
	CNN	73.28	63.17	79.58	73.28	73.43
	LSTM	74.15	64.56	81.43	74.15	74.82
	TabT	**75.59**	64.11	81.68	**75.59**	**75.90**
	FT-T	75.33	63.70	76.92	75.33	75.74

Bold means the best result for that metric for a specific experiment (class/dataset)

- (ii) On `DAT2` FT-T halves the error rate on *macOS X*; the residual confusion *Android-iOS* stems from newer Android builds that clone Apple's TCP-option layout.
- (iii) On `DAT3` most mistakes are *BSD-Linux*, two Unix-like families whose Nmap signatures differ mainly in TCP window-scaling heuristics rarely probed by default.

Operationally, this means an analyst can trust FT-T's *"Windows server"* alarms (≤0.7% FP) but should treat an *iOS vs Android* distinction with caution on heavily proxied Wi-Fi.

Overall, FT-Transformer emerged as the most robust model, consistently outperforming both TabTransformer and traditional ML methods across most scenarios. A visual comparison of the results is provided in Fig. 6, and standardized confusion matrices for the `family` classification are presented in Figs. 7, 8, 9.

Novel contributions. This study pioneers the application of the attention mechanism via Transformer architectures to the OS fingerprinting task, achieving improved outcomes over classic ML models and opening new research directions in advanced deep learning applications for network security.

Reproducibility. The complete code and all results are publicly available under a GNU GPL v3.0 license at https://github.com/rubenpjove/tabularT-OS-fingerprinting.

Key findings and observations. Our experimental evaluation on the three datasets demonstrates that Transformer-based models—especially the FT-Transformer (FT-T)—consistently deliver superior performance in OS fingerprinting tasks.

Comparing original and reproduced results in OS fingerprinting research is challenging due to the lack of accuracy metrics or detailed evaluation data, combination of similar OS versions in the same class, etc. This can artificially inflate accuracy rates, leading to inconsistent identification, where some OS versions are specified in detail while others are grouped broadly. Additionally, this study compares the results of the proposed Tabular Transformer-based methods with prior research in the field.

Table 10 offers a comprehensive comparison of the proposed methods against existing techniques, for each employed dataset (`DAT1`, `DAT2` and `DAT3`) and granularity of the classification (`family`, `major` and `minor`). For each work, we specify the ML technique used.

- **DAT1**: *FT-Transformer outperforms the state-of-the-art method by approximately 12% in F1-score.* This dataset, originally employed in Lastovicka et al. (2020), contains only OS information for `family` classification. While the referenced work compared a novel TLS-based decision tree with select baselines, our FT-T method achieves an F1-score

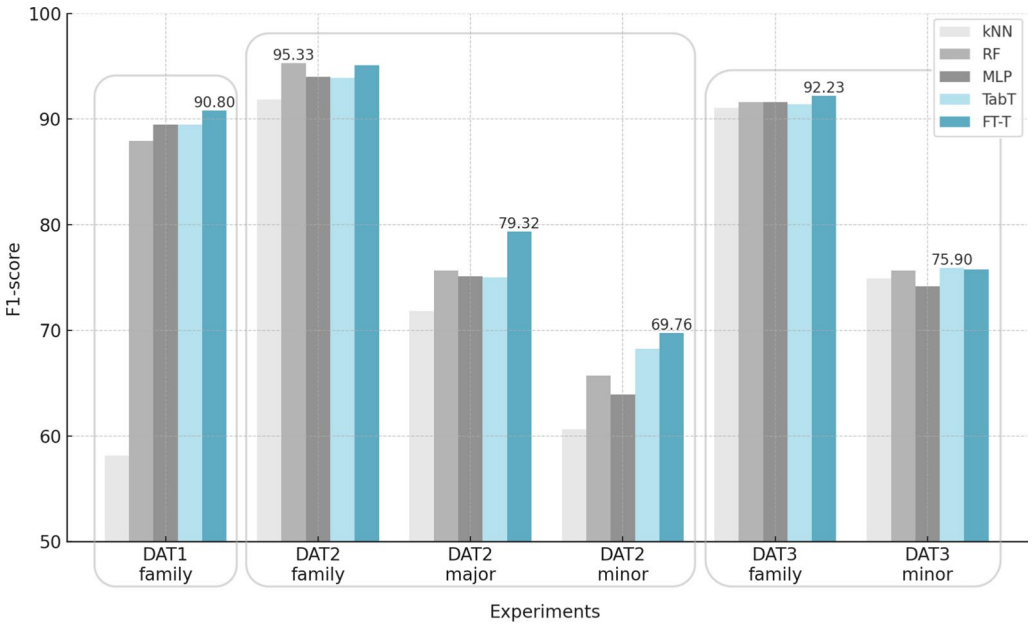


Fig. 6 Comparison of F1-scores (weighted) across tested models for various datasets and classification tasks. Each experiment represents a dataset and granularity of the classification, with the highest performance for each experiment annotated at the top of the corresponding bar

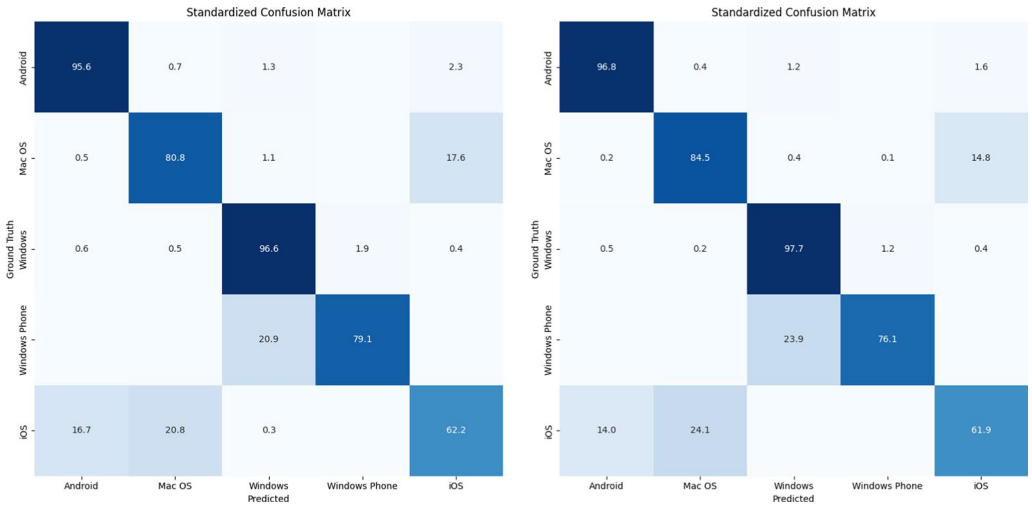


Fig. 7 Standardised confusion matrices for `family` classification in `DAT1` (TabT: *left*, FT-T: *right*)

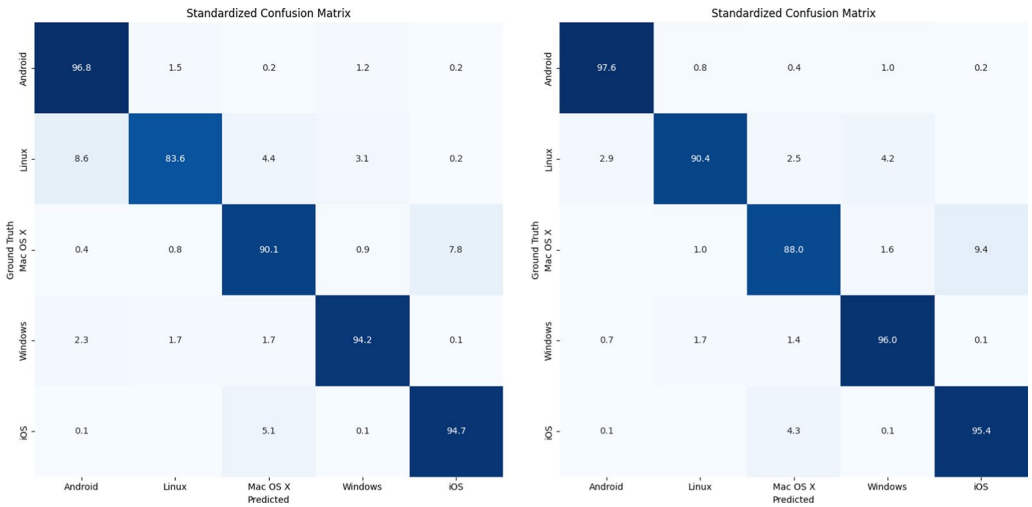


Fig. 8 Standardised confusion matrices for `family` classification in `DAT2` (TabT: *left*, FT-T: *right*)

improvement of around 12%, as shown in the first part of Table 10.

- **DAT2**: *FT-Transformer sets a new benchmark for* `family` *classification, uniquely addresses the* `major` *level, and remains competitive on the challenging* `minor` *task.* Utilized in Lastovicka et al. (2023), `DAT2` supports OS classification at `family`, `major`, and `minor` levels. For `family` classification, FT-T achieves an F1-score of 95.09%, substantially outperforming the second-best model (DT at 81.60%). No prior work has addressed the `major` level, making FT-T the sole and best-performing model in this category. For the `minor` classification, despite its inherent difficulty, FT-T remains competitive with an F1-score of 69.76%. In this latest case, it is worth mentioning that our implementation uses the full `minor` label ("`family-major-minor`", 88 classes) derived exactly as described in Sect. 'Data preparation'. Earlier work (Lastovicka et al. 2023) groups several adjacent minor releases under broader labels (32 classes). The lower value in Table 8 therefore stems from stricter label granularity rather than an implementation problem.
- **DAT3**: *FT-Transformer excels in* `family` *classification while TabTransformer shows the best performance on* `minor` *classification.* Derived from the Nmap (v.7.94) database, `DAT3` supports both `family` and `minor` classifications. In the `family` task, FT-T achieves an F1-score of 92.23%, outperforming the best baseline (MLP). Conversely, for the `minor`

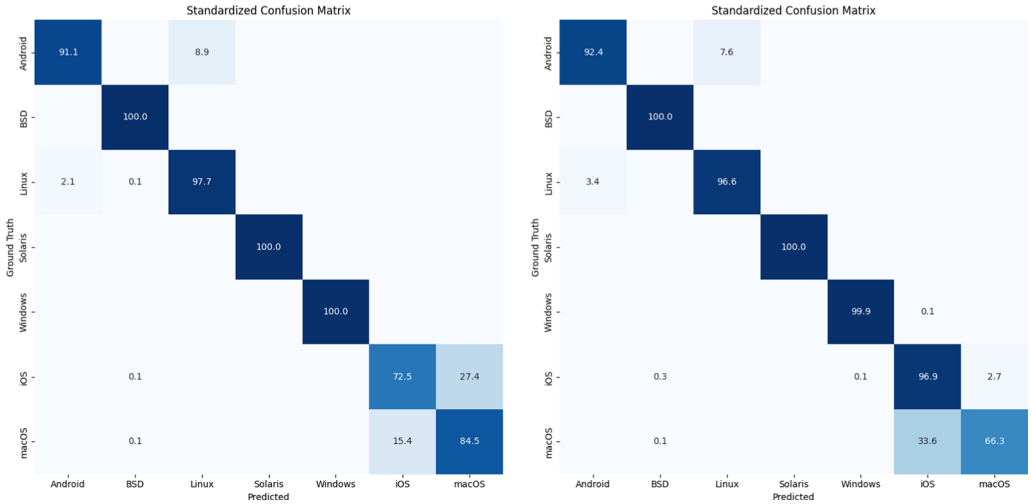


Fig. 9 Standardised confusion matrices for `family` classification in `DAT3` (TabT: *left*, FT-T: *right*)

Table 10 Comparative analysis of proposed methods and previous works for OS fingerprinting across various datasets and classification tasks

Dataset	Class	Source	Model	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
`DAT1`	`family`	Lastovicka et al. (2020)	DT	**93.12**	80.48	77.49	78.96
		Proposed	FT-T	90.69	**91.01**	**90.69**	**90.80**
`DAT2`	`family`	Beverly (2004) in Lastovicka et al. (2023)	Bayes	37.70	49.70	37.60	37.40
		Lastovicka et al. (2018) in Lastovicka et al. (2023)	DT	84.10	83.60	94.30	81.60
		Lastovicka et al. (2020) in Lastovicka et al. (2023)	DT	82.10	81.60	92.40	81.60
		Lippmann et al. (2003) in Lastovicka et al. (2023)	kNN	94.70	–	–	–
		Hulák et al. (2023)	DT/RF	88.00	76.00	–	76.00
		Proposed	FT-T	**95.04**	**95.19**	**95.04**	**95.09**
	`major`	Proposed	FT-T	**79.09**	**80.00**	**79.09**	**79.32**
	`minor`	Beverly (2004) in Lastovicka et al. (2023)	Bayes	0.50	**92.10**	0.60	0.30
		Lastovicka et al. (2018) in Lastovicka et al. (2023)	DT	73.70	81.10	**73.60**	68.50
		Lastovicka et al. (2020) in Lastovicka et al. (2023)	DT	73.40	71.50	73.40	**71.40**
		Lippmann et al. (2003) in Lastovicka et al. (2023)	kNN	**92.10**	–	–	–
		Proposed	FT-T	68.52	72.66	68.52	69.76
`DAT3`	`family`	Baseline	MLP	91.65	93.50	91.65	91.62
		Proposed	FT-T	**92.35**	**93.74**	**92.35**	**92.23**
	`minor`	Baseline	RF	73.92	**81.76**	73.92	75.66
		Proposed	TabT	**75.59**	81.68	**75.59**	**75.90**

Bold means the best result for that metric for a specific experiment (class/dataset)

classification, TabTransformer outperforms the kNN baseline with an F1-score of 75.90%.

The relative behaviour of FT-T, TabT and RF is best understood through the lens of data quantity and feature composition. FT-T places most of its attention on high-cardinality categorical tokens (TLS extension types/lengths, JA3 fingerprint, User-Agent bytes) while TabT relies on a much larger embedding of every categorical column. In `DAT1` and the two finer-grained tasks of `DAT2` these rich categorical cues dominate, giving FT-T a clear edge. Conversely, the `family` split of `DAT2` is line-rate traffic captured on only five web servers: most flows share identical TLS and HTTP metadata, so the task can be solved almost entirely from a handful of transport statistics. Because RF ensembles are known to excel when a small subset of numeric features carries the decision boundary, RF attains a marginally higher F1 (95.33% vs 95.09%). When we ablated the TLS columns or trained both models on smaller random samples, RF kept > 94% F1 even with 25% of the data, whereas FT-T required at least half the observations plus the full feature set to regain its lead—evidence that the isolated RF win stems from feature sparsity rather than an intrinsic limitation of attention-based models.

In practical deployments, these findings suggest a two-tier strategy: use FT-T when at least basic TLS metadata and > 50 k labelled flows are available; fall back to an RF ensemble on feature-poor or very small captures. Nonetheless, all results reflect university-backbone traffic with rule-derived labels; unseen enterprise networks, VPN encapsulation or deliberate obfuscation may lower accuracy by several points. Future work will therefore expand evaluation to heterogeneous vantage points and include adversarial traffic shaping experiments.

Miss-classification patterns in Figs. 7, 8, 9 reinforce this view. Across all datasets the harshest confusions arise between OS pairs that share either a TCP/IP lineage or the same mobile browser stack: Windows Phone is mistaken for Windows 10 in 23.9% of the remaining errors, and iOS devices are confused with macOS laptops in 14–17% of flows. These flows carry virtually indistinguishable SYN sizes and initial window values; only subtle differences in JA3 curves separate them. Transformer heads that include TLS bytes succeed in recovering part of this signal (blue matrices show around 10 pp fewer off-diagonal counts than the TabT counterparts), yet a residual ambiguity remains—a limitation inherent to passive fingerprinting with encrypted traffic.

Overall, the FT-Transformer's consistent superior performance suggests it is particularly adept at extracting informative features from both numerical and categorical data through its attention mechanism.

Computational resources

DL methods become attractive only if their accuracy gains are not offset by prohibitive computational overhead. This section therefore answers three questions:

- (i) How large are the proposed models and how long do they take to train?
- (ii) What is the per-flow inference cost and achievable throughput on typical hardware?
- (iii) How does the accuracy-cost balance compare with classical ML baselines?

End-to-end cost across all tasks

Table 11 and Fig. 10 summarise model size, total training time, and inference latency for every dataset and classification granularity on the proposed Transformer-based models.

- **Model footprint.** FT-T never exceeds 0.5M parameters, whereas TabT ranges from 66 M to over 520 M parameters. The difference is most striking on `DAT2-family`, where FT-T is 2300x smaller.
- **Training effort.** FT-T trains in under five minutes on a single CPU node for all but one scenario; TabT may require more than an hour. Such short training cycles facilitate rapid retraining when new operating-system versions emerge.
- **Inference speed.** CPU-only inference for FT-T is consistently below 0.07 ms per flow (14–32k samples/s). TabT is one order of magnitude slower.

Commodity-hardware study

To examine practical deployment, we repeated a representative experiment (`DAT2-major`) on three simulations of commercial platforms: (i) a desktop workstation (Intel i7-10700, 8 cores), (ii) a server-class CPU node (Intel Xeon 8352Y, 32 Cores), and (iii) the same node with a low-profile NVIDIA T4 GPU. Results are collated in Table 12 and visualised in Fig. 11.

- Even on the desktop, FT-T processes 1.7k flows/s (0.58 ms per flow), already sufficient for small-office edge links.
- On the CPU server, throughput rises to 9.9k flows/s (0.10 ms).
- With the T4 GPU, FT-T exceeds 32k flows/s (0.03 ms), leaving ample headroom for line-rate analysis at 10 Gb/s, where a flow record typically arrives every 0.3 ms.

Accuracy-efficiency trade-off

Compared with the fastest classical ML baseline (MLP), FT-T is slightly slower on CPU but improves

Table 11 Computational resource usage and inference efficiency across datasets, models, and OS classification tasks

Dataset	Class	Model	#Params	Train	Inf./sample (ms)	TP (sps)
DAT1	family	TabT	65.8M	8m04s	0.247	4041
		FT-T	125K	55s	0.071	14 182
DAT2	family	TabT	520.1M	1h01m	0.410	2440
		FT-T	232K	3m06s	0.030	32 854
	major	TabT	131.1M	19m44s	0.091	10 998
		FT-T	496K	4m44s	0.047	21 126
	minor	TabT	131.1M	55m26s	0.222	4506
		FT-T	494K	4m40s	0.048	20 963
DAT3	family	TabT	134.4M	3m26s	0.077	13 044
		FT-T	66K	3m06s	0.060	16 613
	minor	TabT	134.6M	1h58m	0.101	9887
		FT-T	101K	25m51s	0.032	30 841

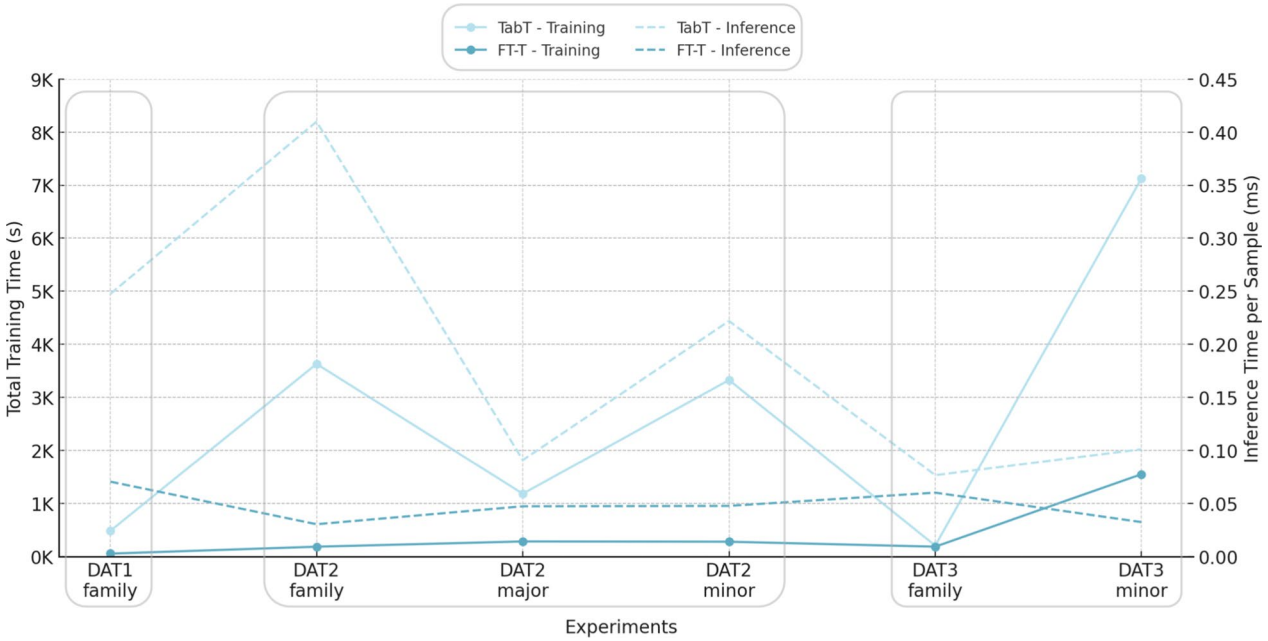


Fig. 10 Comparison of total training time and inference time per sample across tested models for various datasets and classification tasks

Table 12 Performance comparison of inference and training on commercial hardware platforms (DAT2 - major)

Model	Platform	CPU	GPU	Train	Inf./sample (ms)	TP (sps)
FT-T	Local workstation	Intel i7-10700, 8 cores	No	3h28m	0.578	1730
kNN	FinisTerrae III	Intel Xeon 8352Y, 32 cores	No	2.4s	1.482	675
RF	FinisTerrae III	Intel Xeon 8352Y, 32 cores	No	2m06s	0.077	12 987
MLP	FinisTerrae III	Intel Xeon 8352Y, 32 cores	No	26m02s	0.011	90,909
FT-T	FinisTerrae III	Intel Xeon 8352Y, 32 cores	No	1h02m	0.101	9910
FT-T	FinisTerrae III	Intel Xeon 8352Y, 32 cores	NVIDIA Tesla T4	3m59s	0.031	32,290

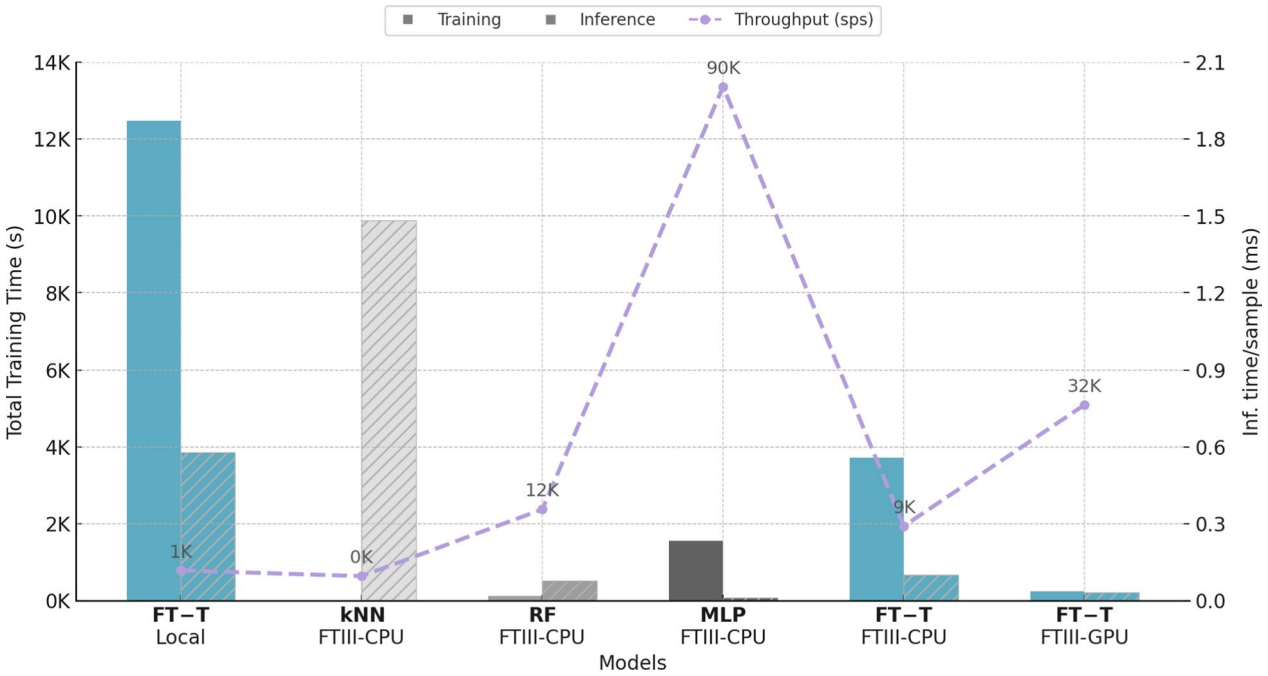


Fig. 11 Performance comparison of inference and training on commercial hardware platforms (`DAT2-major`)

F1 by at least five points on every task. Relative to the ubiquitous kNN, FT-T is an order of magnitude quicker and markedly more accurate. These results confirm that the proposed FT-T delivers state-of-the-art performance without imposing unacceptable resource demands.

Deployability outlook

The final FT-T model occupies less than 5 MB, fits comfortably on edge sensors and requires only commodity hardware for real-time inference. The heavier TabT variant offers no accuracy advantage and is therefore unnecessary in practice. Taken together, the measurements demonstrate that Transformer-based OS fingerprinting is feasible for production-level monitoring without specialised supercomputing facilities.

Ablation study

To quantify how much each input modality contributes to the overall gain of the FT-Transformer, we performed an ablation in which the model was trained on:

- (i) Both categorical and numerical features (full setting),
- (ii) Categorical features only, and
- (iii) Numerical features only.

All other hyper-parameters, training epochs and random seeds were kept identical to the main experiment on the `DAT2-major` task. The single-seed results are summarised in Table 13.

The full model attains an F1 of 80.83%. Removing the numerical variables reduces performance by 11.84 percentage points (pp), indicating that purely categorical information, although highly informative, does not capture the full variability needed for precise version classification. Conversely, excluding the categorical tokens and feeding only continuous fields lowers F1 by 20.63 pp relative to the complete configuration and by 8.79 pp compared with the categorical-only version.

These findings suggest that (a) categorical descriptors such as protocol identifiers and TLS byte arrays provide the primary discriminative signal, and (b) numerical attributes-including packet lengths and timing statistics-supply complementary fine-grained detail that the attention mechanism exploits to refine the decision boundary. The marked improvement when both modalities are present confirms that the FT-Transformer's joint handling

Table 13 FT-Transformer ablation study over input feature types (single-seed results; F1 in %). Δ columns report percentage-point (pp) differences

Features	F1 (%)	Δ vs. Cat (pp)	Δ vs. Num (pp)
cat + num	**80.83**	**+11.84**	**+20.63**
cat	68.99	–	+8.79
num	60.20	−8.79	–

of heterogeneous features is critical to its superior performance.

Interpretability

Beyond raw accuracy, an operational classifier must reveal which packet attributes underpin its predictions, especially for network security tasks such as OS fingerprinting. We therefore inspected the self-attention patterns learned by the FT-Transformer on the `family` task of `DAT1`, whose concise feature set supports clear visual analysis.

For every encoder layer and head we extracted the attention matrices, averaged them across the entire test split, and projected the resulting scores back onto the original feature space. Two complementary statistics are reported for each feature: (i) the *average attention weight* (mean of all normalised scores) and (ii) the *summed attention weight* (mean multiplied by the sequence length). These values are displayed in Fig. 12.

The model focuses chiefly on three TLS handshake attributes—"`TLS Extension Lengths`", "`TLS Extension Types`" and "`TLS Elliptic Curves`"—which together account for more than half of the total attention mass. This confirms long-standing evidence that subtle variations in the "`ClientHello` message are strong indicators of the underlying operating-system family. Among transport-layer cues, "`SYN size`" and "`TCP SYN TTL`" receive the next largest weights, echoing their prominence in classical tools such as p0f and Nmap. By contrast, legacy signals like the initial "`TCP window size`" attract little weight, suggesting that the network has learned to discount features that are either weakly discriminative or frequently normalised by modern stacks.

Overall, the attention analysis shows that the classifier relies on features that are both well known to characterise OS behaviour and straightforward to capture at perimeter sensors, which supports its suitability for real-world deployment. Extending this study with additional attribution techniques such as SHAP or Integrated Gradients forms part of our future work agenda.

Adversarial robustness study

Real-world adversaries can attempt to evade fingerprinting by subtly altering selected header fields or by introducing noise into packet characteristics through tools such as fragrouter or Nping's fuzzing modes. To evaluate how sensitive the best proposed model, FT-T, is to such manipulations, we simulated a representative evasion scenario on a relevant setting in our benchmark—`DAT2` at the `major` granularity—where classical models already exhibit a marked performance drop (Table 8).

We assume an attacker who can tamper with a limited subset of easily modifiable transport-layer fields while leaving the remainder of the flow unchanged. Following common obfuscation practice, we selected three high-impact features that feature prominently in both traditional rulesets and our feature-importance analysis (Fig. 12 in Sect. 'Interpretability'): "`TCP SYN Size`", "`TCP Win Size`", and "`TCP SYN TTL`". For each noise level $\alpha \in 0.05, 0.10, 0.20, 0.50$ we added zero-mean Gaussian noise with standard deviation $\alpha\sigma_f$, where

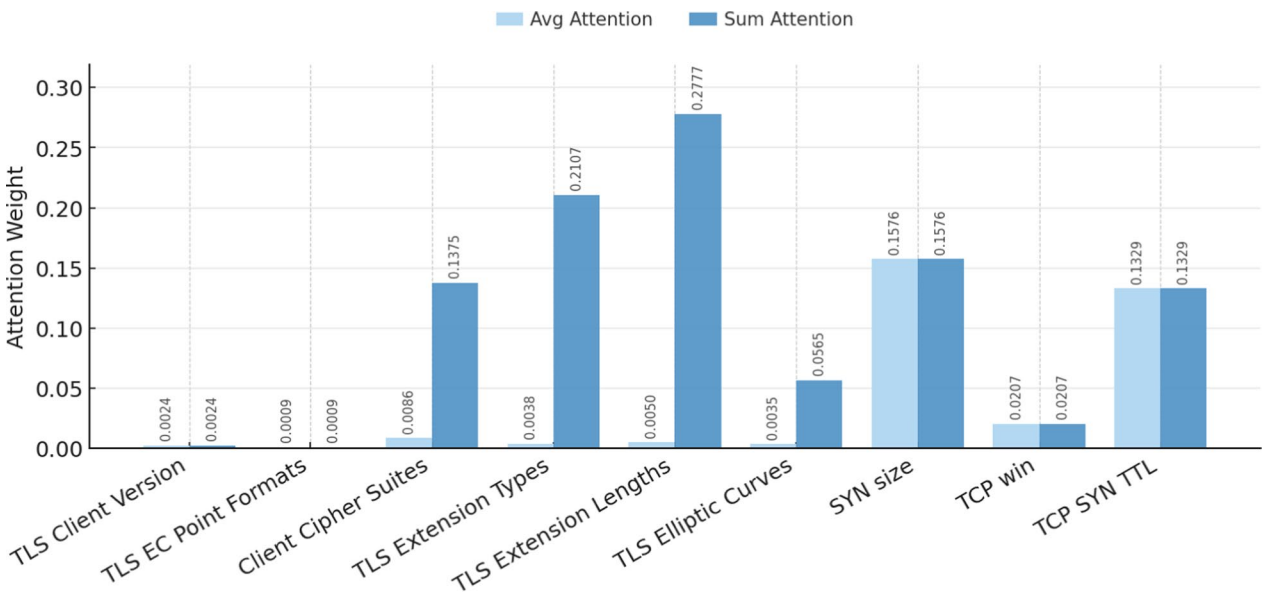


Fig. 12 Average and summed FT-Transformer attention per original feature, for `family` classification in `DAT1`

σ_f is the empirical standard deviation of feature f in the training partition. Perturbations were applied to all samples of the held-out test split; the model was not fine-tuned afterwards, mirroring a black-box evasion attempt.

The impact on F1 is summarised in Table 14. A mild perturbation of 5% causes only a 1.7% relative decrease (−1.3 pp), indicating that FT-T retains a high margin of correct classification even when canonical fingerprinting cues are slightly distorted. Performance degrades gracefully as noise grows: at the extreme 50% level—well beyond the variability introduced by legitimate stacks—the F1 loss is still below 10% relative (−7.5 pp). Crucially, the model continues to outperform all classical baselines even under the heaviest perturbation (reported in Table 8).

These findings suggest that the attention mechanism endows FT-T with a degree of feature redundancy: when the most obvious cues are corrupted, the network can fall back on secondary correlations captured during training, thereby limiting performance degradation. While Gaussian noise is only a first-order proxy for deliberate obfuscation, the experiment provides evidence that Transformers are at least as robust as and often more robust than classical ML approaches in the face of moderate evasion.

Nevertheless, dedicated adversarial attacks that optimise perturbations jointly across multiple fields (e.g. gradient-based methods or flow re-synthesis) remain an open challenge. Future work will extend this analysis to (i) protocol-conformant adversarial examples, (ii) adaptive attackers that observe the classifier's output, and (iii) defensive fine-tuning strategies such as adversarial training or feature randomisation.

Conclusion & future work

In this work, we introduced the novel application of Transformer architectures adapted for tabular data to the task of OS fingerprinting. By leveraging both the TabTransformer and FT-Transformer models, our study bridges the gap between traditional rule-based methods and modern DL approaches in the cybersecurity domain. The experimental evaluation across multiple diverse datasets demonstrated that attention-based models can effectively capture complex interactions within network traffic, yielding improved OS classification performance over conventional ML baselines and previous works.

Our results indicate that the FT-Transformer, in particular, offers significant advantages in terms of accuracy and robustness when classifying OSs at varying levels of granularity and network environments. The inherent self-attention mechanisms allow the model to learn intricate feature dependencies, which is crucial for processing heterogeneous and dynamic network traffic data. This performance gain, as compared to previous works and traditional models such as k-Nearest Neighbours, Random Forests, and Multi-Layer Perceptron, establishes a new benchmark for OS fingerprinting tasks and highlights the potential of advanced DL techniques in this area.

In future work, we aim to explore several directions to further enhance the application of Tabular Transformer architectures for OS fingerprinting. First, expanding the range of datasets. Although the selected datasets are, in principle, representative of modern network environments, additional data from real-world deployments would be beneficial to further validate the models under a wider range of conditions. Additionally, we plan to investigate the impact of incorporating hybrid DL approaches, such as combining Transformers with Graph Neural Networks to better capture the temporal-spatial relationships in network traffic data. Furthermore, we intend to evaluate the performance of these architectures on new or modified OS versions, directly comparing them with traditional tools such as Nmap, to assess their robustness and practical applicability in dynamic environments. Finally, we aim to apply transfer learning by pre-training on larger, related datasets to further refine performance across diverse and dynamic network conditions.

Moreover, the integration of the developed model into real-world cybersecurity tools, such as Nmap and various intrusion detection systems, could substantially enhance their versatility and operational effectiveness. We believe that our approach is not only beneficial for OS fingerprinting but also holds promise for broader applications within network security, including intrusion detection and traffic anomaly detection, thereby potentially yielding a significant industrial impact.

Table 14 Impact of Gaussian noise injection on FT-T model (`DAT2 - major`). Δ columns report percentage-point (pp) and relative (%) differences with respect to the F1 original model

Noise level	F1 (%)	Δ (pp)	Δ (%)
Original	78.90	–	–
0.05	77.60	−1.30	−1.7
0.10	76.10	−2.80	−3.6
0.20	74.10	−4.80	−6.1
0.50	71.40	−7.50	−9.5

Acknowledgements

This work was made possible through the access granted by the Galician Supercomputing Center (CESGA) to its supercomputing infrastructure. The supercomputer FinisTerrae III and its permanent data storage system have been funded by the Spanish Ministry of Science and Innovation, the Galician Government and the European Regional Development Fund (ERDF).

Authors' contributions
Rubén Pérez-Jove Study conception, experiments, and manuscript drafting; Cristian R. Munteanu Supervision, review, and final approval; Alejandro Pazos Supervision, review, and final approval; Jose Vázquez-Naya Supervision, review, and final approval.

Funding
This work was supported by the grant ED431C 2022/46 - Competitive Reference Groups GRC - funded by "Xunta de Galicia" (Spain). This work was also supported by CITIC, as a center accredited for excellence within the Galician University System and a member of the CIGUS Network, which receives subsidies from the Department of Education, Science, Universities, and Vocational Training of the "Xunta de Galicia". Additionally, CITIC is co-financed by the EU through the FEDER Galicia 2021–27 operational program (Ref. ED431G 2023/01). This work was also supported by the "Formación de Profesorado Universitario" (FPU) grant from the Spanish Ministry of Universities to Rubén Pérez Jove (Grant FPU22/04418).

Data availability
The datasets and materials supporting the conclusions of this article are available at https://github.com/rubenpjove/tabularT-OS-fingerprinting. Additional information not included in the repository can be obtained from the corresponding author upon reasonable request.

Declarations

Conflict of interest
The authors declare that they have no conflict of interest.

Received: 14 February 2025 Accepted: 25 September 2025
Published: 10 June 2026

References

Aksoy A, Gunes MH (2016) Operating system classification performance of TCP/IP protocol headers. In: 2016 IEEE 41st conference on local computer networks workshops (LCN workshops), pp 112–120. https://doi.org/10.1109/LCN.2016.036
Al-Shehari T, Shahzad F (2014) Improving operating system fingerprinting using machine learning techniques. Int J Comput Theory Eng 6:57–62. https://doi.org/10.7763/IJCTE.2014.V6.837
Arkin O (2002) A remote active OS fingerprinting tool using ICMP. Login Mag USENIX Sage 27(2):14–19
Auffret P (2010) SinFP, unification of active and passive operating system fingerprinting. J Comput Virol 6(3):197–205
Barath J, Liska M (2021) Use of data mining techniques for network data analysis. In: 2021 communication and information technologies (KIT), pp 1–6. https://doi.org/10.1109/KIT52904.2021.9583755
Barut O, Luo Y, Li P, Zhang T (2023) R1dit: privacy-preserving malware traffic classification with attention-based neural networks. IEEE Trans Netw Serv Manag 20(2):2071–2085. https://doi.org/10.1109/TNSM.2022.3211254
Beddoe M. (2000) The Siphon project: the passive network mapping tool. https://github.com/unmarshal/siphon
Beverly R (2004) A robust classifier for passive TCP/IP fingerprinting. In: Passive and active network measurement, pp 158–167. https://doi.org/10.1007/978-3-540-24668-8_16
Breiman L (2001) Random forests. Mach Learn 45(1):5–32
Bub D, Hartmann L, Bozakov Z, Wendzel S (2022) Towards passive identification of aged android devices in the home network. In: EICC 2022: proceedings of the European interdisciplinary cybersecurity conference, pp 17–20. https://doi.org/10.1145/3528580.3528584
Cover TM, Hart PE (1967) Nearest neighbor pattern classification. IEEE Trans Inf Theory 13(1):21–27
Dietmüller A, Ray S, Jacob R, Vanbever L (2022) A new hope for network model generalization. In: Proceedings of the 21st ACM workshop on hot topics in networks. HotNets '22, pp 152–159. https://doi.org/10.1145/3563766.3564104
Dosovitskiy A, Beyer L, Kolesnikov A, Weissenborn D, Zhai X, Unterthiner T, Dehghani M, Minderer M, Heigold G, Gelly S, Uszkoreit J, Houlsby N (2021) An image is worth 16 x 16 words: transformers for image recognition at scale. arXiv. https://doi.org/10.48550/arXiv.2010.11929
Fan X, Gou G, Kang C, Shi J, Xiong G (2019) Identify OS from encrypted traffic with TCP/IP stack fingerprinting. In: 2019 IEEE 38th international performance computing and communications conference (IPCCC), pp 1–7. https://doi.org/10.1109/IPCCC47392.2019.8958772
Fjellskål EB, Wysocki K (2009) PRADS–passive real-time asset detection system. https://github.com/gamelinux/prads
Galicia CC. FinisTerrae III user guide CESGA technical documentation 1.0.0 documentation. https://cesga-docs.gitlab.io/ft3-user-guide/index.html. Accessed 12 Jul 2024
Gorishniy Y, Rubachev I, Khrulkov V, Babenko A (2021) Revisiting deep learning models for tabular data. arXiv. https://doi.org/10.48550/arXiv.2106.11959
Gurary J, Zhu Y, Bettati R, Guan Y (2016) Operating system fingerprinting. In: Wang C, Gerdes RM, Guan Y, Kasera SK (eds) Digital fingerprinting. Springer, Cham, pp 115–139. https://doi.org/10.1007/978-1-4939-6601-1_7
Guthula S, Battula N, Beltiukov R, Guo W, Gupta A (2023) netFound: foundation model for network security. arXiv. https://doi.org/10.48550/arXiv.2310.17025
Hagos DH, Loland M, Yazidi A, Kure O, Engelstad PE (2020) Advanced passive operating system fingerprinting using machine learning and deep learning. In: 2020 29th international conference on computer communications and networks (ICCCN), pp 1–11. https://doi.org/10.1109/ICCCN49398.2020.9209694
Hagos DH, Yazidi A, Kure O, Engelstad PE (2021) A machine-learning-based tool for passive OS fingerprinting with TCP variant as a novel feature. IEEE Internet Things J 8(5):3534–3553. https://doi.org/10.1109/JIOT.2020.3024293
Han K, Wang Y, Chen H, Chen X, Guo J, Liu Z, Tang Y, Xiao A, Xu C, Xu Y, Yang Z, Zhang Y, Tao D (2023) A survey on vision transformer. IEEE Trans Pattern Anal Mach Intell 45(1):87–110. https://doi.org/10.1109/TPAMI.2022.3152247
Hochreiter S, Schmidhuber J (1997) Long short-term memory. Neural Comput 9(8):1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735
Huang X, Khetan A, Cvitkovic M, Karnin Z (2020) TabTransformer: tabular data modeling using contextual embeddings. arXiv. https://doi.org/10.48550/arXiv.2012.06678
Hulák M, Bartos V, Cejka T (2023) Evaluation of passive OS fingerprinting methods using TCP/IP fields. In: 2023 8th international conference on smart and sustainable technologies, SpliTech 2023. https://doi.org/10.23919/SpliTech58164.2023.10192974
Kollmann E (2018) Satori. https://github.com/xnih/satori
Lastovicka M, Dufka A, Komárková J (2018) Machine learning fingerprinting methods in cyber security domain: which one to use? In: 2018 14th international wireless communications & mobile computing conference (IWCMC), pp 542–547. https://doi.org/10.1109/IWCMC.2018.8450406
Lastovicka M, Husák M, Velan P, Jirsík T, Celeda P (2023) Dataset–passive operating system fingerprinting revisited–network flows dataset. https://doi.org/10.5281/ZENODO.7635138
Lastovicka M, Husák M, Velan P, Jirsík T, Celeda P (2023) Passive operating system fingerprinting revisited: evaluation and current challenges. Comput Netw 229:109782. https://doi.org/10.1016/j.comnet.2023.109782
Lastovicka M, Jirsik T, Celeda P, Spacek S, Filakovsky D (2018) Passive OS fingerprinting methods in the jungle of wireless networks. In: NOMS 2018–2018 IEEE/IFIP network operations and management symposium, pp 1–9. https://doi.org/10.1109/NOMS.2018.8406262
Lastovicka M, spacek S, Velan P, Celeda P (2020) Using TLS fingerprints for OS identification in encrypted traffic. In: NOMS 2020–2020 IEEE/IFIP network operations and management symposium, pp 1–6. https://doi.org/10.1109/NOMS47738.2020.9110319
LeCun Y, Boser B, Denker JS, Henderson D, Howard RE, Hubbard W, Jackel LD (1989) Backpropagation applied to handwritten zip code recognition. Neural Comput 1(4):541–551. https://doi.org/10.1162/neco.1989.1.4.541
Le F, Srivatsa M, Ganti R, Sekar V (2022) Rethinking data-driven networking with foundation models: challenges and opportunities. In: Proceedings of the 21st ACM workshop on hot topics in networks. HotNets '22, pp 188–197. https://doi.org/10.1145/3563766.3564109

Le F, Wertheimer D, Calo S, Nahum E (2022) NorBERT: NetwOrk representations through BERT for network analysis & management. In: 2022 30th international symposium on modeling, analysis, and simulation of computer and telecommunication systems (MASCOTS), pp 25–32. https://doi.org/10.1109/MASCOTS56607.2022.00012
Lin X, Xiong G, Gou G, Li Z, Shi J, Yu J (2022) ET-BERT: a contextualized datagram representation with pre-training transformers for encrypted traffic classification. In: Proceedings of the ACM web conference 2022, pp 633–642. https://doi.org/10.1145/3485447.3512217
Lippmann R, Fried D, Piwowarski K, Streilein W (2003) Passive operating system identification from TCP/IP packet headers. In: Workshop on data mining for computer security, vol 40
Li J, Wei Z, Chen S (2023) Passive OS identification in imbalanced dataset. In: 2023 International Conference on Electrical, Computer and Energy Technologies (ICECET), pp 1–6. https://doi.org/10.1109/ICECET58911.2023.10389569
Louis A, Louppe G (2020) NetBERT: a pre-trained language representation model for computer networking. Master's thesis, Université de Liège, Liège, Belgique. https://matheo.uliege.be/handle/2268.2/9060. Accessed 04 Sept 2023
Martin L, Stanislav S, Petr V, Pavel C (2019) Dataset-using TLS fingerprints for OS identification in encrypted traffic. Zenodo. https://doi.org/10.5281/zenodo.3461771
Matousek J, Wagner U (2014) On Gromov's method of selecting heavily covered points. Discret Comput Geom 52(1):1–33. https://doi.org/10.1007/s00454-014-9584-7
Meng X, Lin C, Wang Y, Zhang Y (2023) NetGPT: Generative pretrained transformer for network traffic. arXiv. https://doi.org/10.48550/arXiv.2304.09513
Millar K, Cheng A, Chew HG, Lim C-C (2020) Operating system classification: a minimalist approach. In: 2020 international conference on machine learning and cybernetics (ICMLC), pp 143–150. https://doi.org/10.1109/ICMLC51923.2020.9469571
NetGrab (2012) Netsleuth. http://netgrab.co.uk/netsleuth/
Nmap Project: Chapter 8. Remote OS detection. https://nmap.org/book/osdetect.html. Accessed 14 Jul 2024
Nmap Project: Nmap: the Network Mapper. https://nmap.org/. Accessed 5 Aug 2022
Ornaghi A (2001) Valleri, Marco, Escobar, Emilio, Costamagna, Gianfranco, Koeppe, Alexander, Abdulkadir. Ettercap Project, Ali
Osanaiye OA, Dlodlo M (2015) TCP/IP header classification for detecting spoofed DDoS attack in cloud environment. In: IEEE EUROCON 2015–international conference on computer as a tool (EUROCON), pp 1–6. https://doi.org/10.1109/EUROCON.2015.7313736
Pérez Jove R, Pazos A, Vázquez Naya J (2024) Towards TabTransformer-based operating system fingerprinting: a preliminary approach using the Nmap database. In: Jornadas Nacionales de investigación en ciberseguridad (JNIC) (9.2024.Sevilla), pp 326–331
Pérez-Jove R, Munteanu CR, Sierra AP, Vázquez-Naya JM (2021) Applying artificial intelligence for operating system fingerprinting. Eng Proc 7:51. https://doi.org/10.3390/engproc2021007051
Pérez-Jove R, Munteanu CR, Dorado J, Pazos A, Vázquez-Naya J (2023) Operating system fingerprinting tool based on classical machine learning algorithms. In: 2023 JNIC cybersecurity conference (JNIC), pp 1–8. https://doi.org/10.23919/JNIC58574.2023.10205734
Project N Nmap OS DB–Revision 38950: /nmap-releases/nmap-7.94. https://svn.nmap.org/nmap-releases/nmap-7.94/nmap-os-db Accessed 14 Jul 2024
Qian C, Li X, Wang Q, Zhou G, Shao H (2024) NetBench: a large-scale and comprehensive network traffic benchmark dataset for foundation models. In: 2024 IEEE international workshop on foundation models for cyber-physical systems & Internet of Things (FMSys), pp 20–25. https://doi.org/10.1109/FMSys62467.2024.00008
Qu J, Ma X, Li J (2024) TrafficGPT: breaking the token barrier for efficient long traffic analysis and generation. arXiv. https://doi.org/10.48550/arXiv.2403.05822
Ray S (2022) Advancing packet-level traffic predictions with transformers. Master thesis, ETH Zurich, Networked Systems. https://doi.org/10.3929/ethz-b-000569234. https://www.research-collection.ethz.ch/handle/20.500.11850/569234 Accessed 23 Sept 2024
Rumelhart DE, Hinton GE, Williams RJ (1986) Learning internal representations by error propagation. MIT Press, Cambridge
Rumelhart DE, Hinton GE, Williams RJ (1986) Learning representations by back-propagating errors. Nature 323(6088):533–536. https://doi.org/10.1038/323533a0
Salah S, Abu Alhawa M, Zaghal R (2022) Desktop and mobile operating system fingerprinting based on IPv6 protocol using machine learning algorithms. Int J Secur Netw 17(1):1–12. https://doi.org/10.1504/IJSN.2022.122543
Shah S (2003) Http fingerprinting and advanced assessment techniques. BlackHat Asia
Shamsi Z, Cline DBH, Loguinov D (2021) Faulds: a non-parametric iterative classifier for internet-wide OS fingerprinting. IEEE/ACM Trans Netw 29(5):2339–2352. https://doi.org/10.1109/TNET.2021.3088333
Shen C, Liu C, Tan H, Wang Z, Xu D, Su X (2018) Hybrid-augmented device fingerprinting for intrusion detection in industrial control system networks. IEEE Wirel Commun 25(6):26–31. https://doi.org/10.1109/MWC.2017.1800132
Torre Vico R, MagÁn-Carrión R, Rodríguez-Gómez RA (2024) Exploring the use of LLMs to understand network traces. In: International joint conferences, pp 122–131. https://doi.org/10.1007/978-3-031-75016-8_12
Van Langendonck L, Castell-Uroz I, Barlet-Ros P (2024) Towards a graph-based foundation model for network traffic analysis. arXiv. https://doi.org/10.48550/arXiv.2409.08111
Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser L, Polosukhin I (2017) Attention is all you need. arXiv. https://doi.org/10.48550/arXiv.1706.03762
Vymlátil M (2014) Detection of operation systems in network traffic using IPFIX. Brno University of Technology, Brno
Wang P (2024) lucidrains/tab-transformer-pytorch
Wang Q, Qian C, Li X, Yao Z, Shao H (2024) Lens: a foundation model for network traffic in cybersecurity. arXiv. https://doi.org/10.48550/arXiv.2402.03646
Zalewski M (2016) p0f v3. https://lcamtuf.coredump.cx/p0f3/#/p0f.shtml. Accessed 05 Aug 2022
Zhang D, Wang Q, Wei Z, Chen S (2022) An operating system identification method based on active learning. In: International conference on electrical, computer, and energy technologies, ICECET 2022. https://doi.org/10.1109/ICECET55527.2022.9873443
Zhao R, Huang Y, Deng X, Xue Z, Li J, Huang Z, Wang Y (2021) Flow transformer: a novel anonymity network traffic classifier with attention mechanism. In: 2021 17th international conference on mobility, sensing and networking (MSN), pp 223–230. https://doi.org/10.1109/MSN53354.2021.00045
Zhao R, Zhan M, Deng X, Wang Y, Wang Y, Gui G, Xue Z (2023) Yet another traffic classifier: a masked autoencoder based traffic transformer with multi-level flow representation. In: Proceedings of the 37th AAAI conference on artificial intelligence, AAAI 37, pp 5420–5427

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.